

Sistemi Operativi
Unità 5: I processi
Aspetti Teorici

Martino Trevisan
Università di Trieste
Dipartimento di Ingegneria e Architettura

Argomenti

1. Concetto di processo
2. Stati di un processo
3. Process Control Block
4. Scheduling
5. Algoritmi di Scheduling

Concetto di processo

Concetto di processo

- Un elaboratore svolge uno o più compiti
 - Esempio: Controllare la temperatura di una stanza
- Un compito si svolge tramite un procedimento formale detto **algoritmo**
- Un programma implementa un algoritmo tramite istruzioni in linguaggio macchina
 - Può essere scritto in un linguaggio di programmazione e *compilato*
- Un **processo** è un programma in esecuzione

Concetto di processo

Inizialmente, ogni elaboratore eseguiva un programma per volta.

- Caricato all'avvio del sistema
- Oppure eseguiti sequenzialmente (batch processing)

I sistemi moderni hanno un SO che permette di eseguire più processi in contemporanea

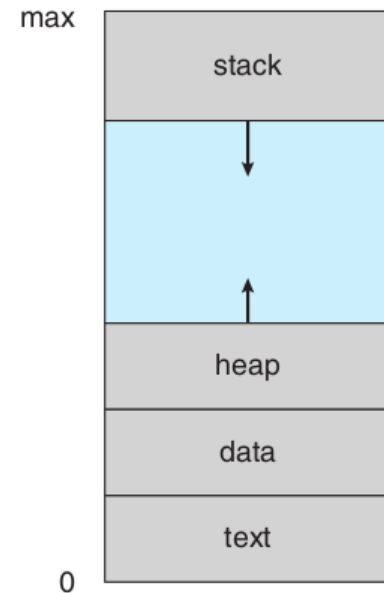
- Le risorse del sistema sono gestite dal SO che le mette a disposizione tramite le System Call
- Il SO gestisce l'esecuzione dei processi: **scheduling**

Concetto di processo

Un processo risiede **in memoria**

La struttura di un processo in memoria è generalmente suddivisa in più sezioni.

- **Sezione di testo:** contiene il codice eseguibile
- **Sezione dati:** contiene le variabili globali
- **Stack:** memoria temporaneamente utilizzata durante le chiamate di funzioni
- **Heap:** memoria allocata dinamicamente durante l'esecuzione del programma



Concetto di processo

Processo *init*

Nei moderni SO, all'avvio del sistema viene avviato un processo fondamentale detto **init**

- Eseguito fino allo shutdown

Il processo *init* avvia altri processi in background:

- Per gestire periferiche: rete, antivirus
- Per creare l'interfaccia grafica della GUI o del terminale

Un processo avviato in background da *init* é detto **Servizio**

- Formati e comandi diversi tra distribuzioni Linux per gestirli

Comandi: `service` o `systemctl`

Concetto di processo

Processi utente

L'utente può creare dei processi per svolgere i propri compiti

- Browser
- Editor
- Programmi server: server Web, server DNS

Nota: anche essi "discendono" da `init`, in quanto è esso che avvia l'interfaccia grafica o il terminale

Concetto di processo

Il SO mette a disposizione delle System Call per:

- Creare nuovi processi
- **Sincronizzazione:** Attendere il completamento di altri processi per coordinare un compito complesso

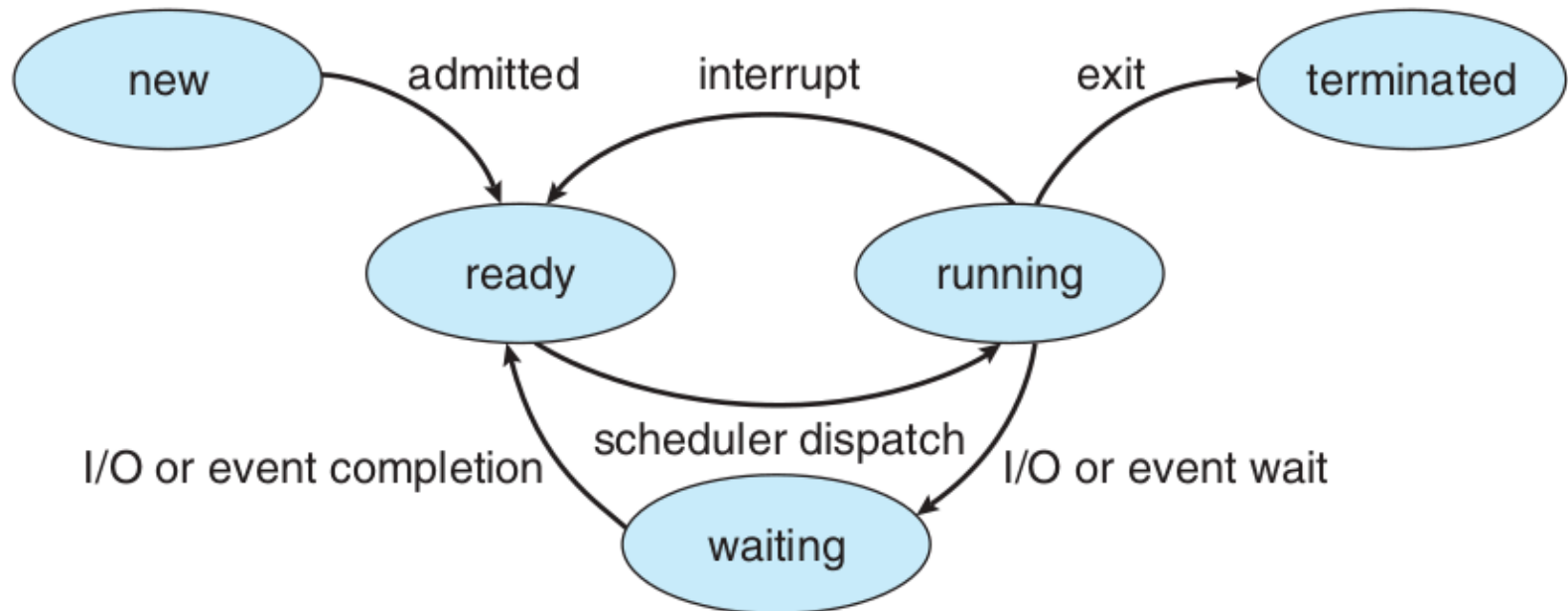
I processi sono identificati dal un **PID**

- Il processo init ha PID **1** per definizione

Stati di un processo

Stati di un processo

Un processo si può trovare in diversi stati



Stati di un processo

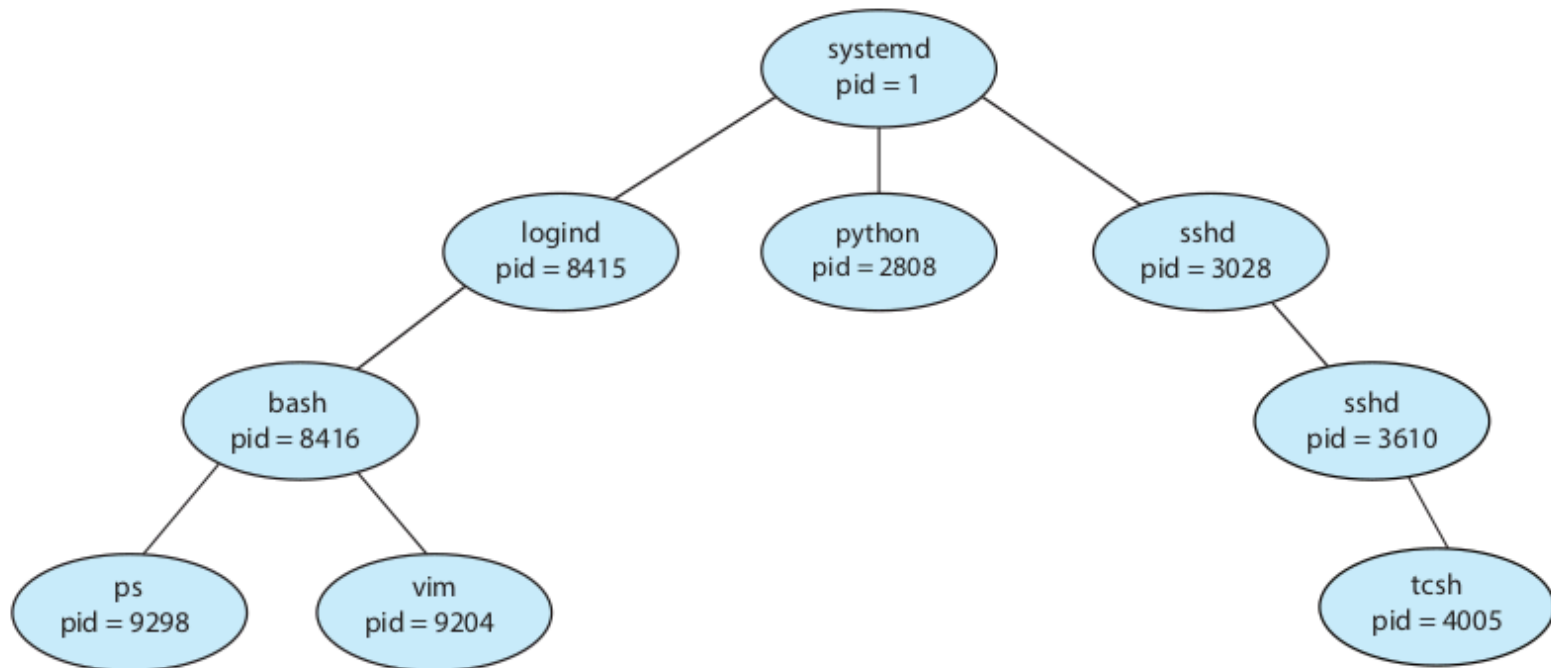
Usando le System Call, un processo può creare un altro processo

- Il processo generato è **figlio** del processo generante
- Si crea un **albero dei processi**
- Se il processo padre termina, i figli **NON** vengono terminati
- I processi *senza padre* diventano figli del processo *init*

Stati di un processo

Esempio di albero dei processi:

Nota: su alcuni sistemi `init` è rinominato `systemd`



Process Control Block

Process Control Block

Ogni processo è rappresentato nel sistema operativo da un blocco di controllo (process control block, PCB) contenente le informazioni connesse

- **Stato del processo:** nuovo, pronto, esecuzione, attesa, arresto
- **Program counter:** indirizzo della successiva istruzione da eseguire
- **Registri della CPU:** permettono di interrompere il processo

process state
process number
program counter
registers
memory limits
list of open files
...

Process Control Block

- **Informazioni di scheduling:** priorità, risorse consumate
- **Informazioni sulla gestione della memoria:** puntatori alle varie zone di memoria
- **Informazioni di I/O:** file aperti, operazioni in attesa, ecc...

process state
process number
program counter
registers
memory limits
list of open files
...

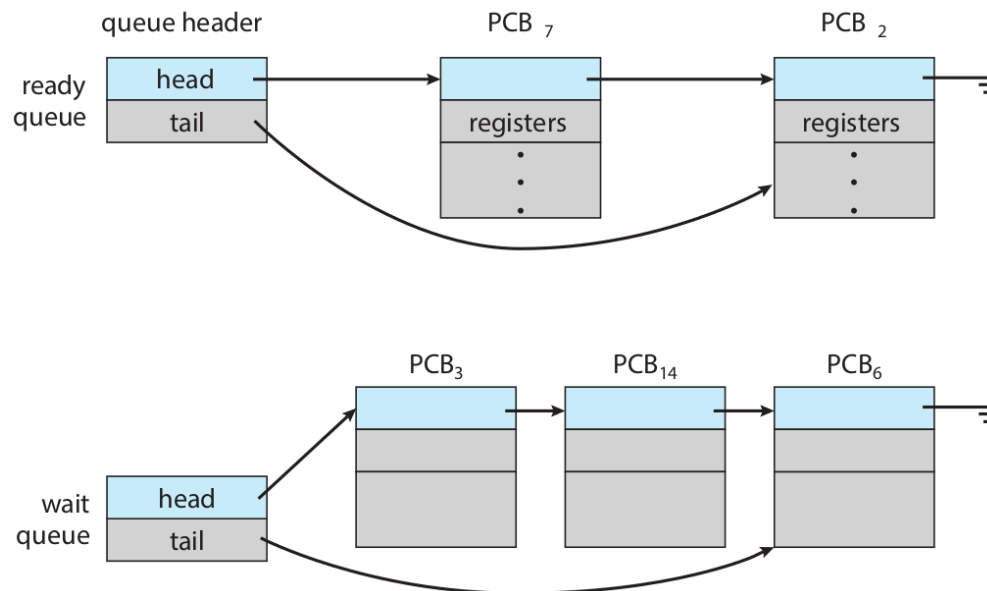
Scheduling

Scheduling

Lo **scheduler dei processi** seleziona un processo da eseguire dall'insieme di quelli disponibili

- Mantiene una coda dei processi pronti
- Mantiene una coda dei processi in attesa di evento.

Esempio: completare un'azione di I/O



Scheduling

I processi possono essere classificati in base al tipo di carico che generano e il collo di bottiglia che li limita.

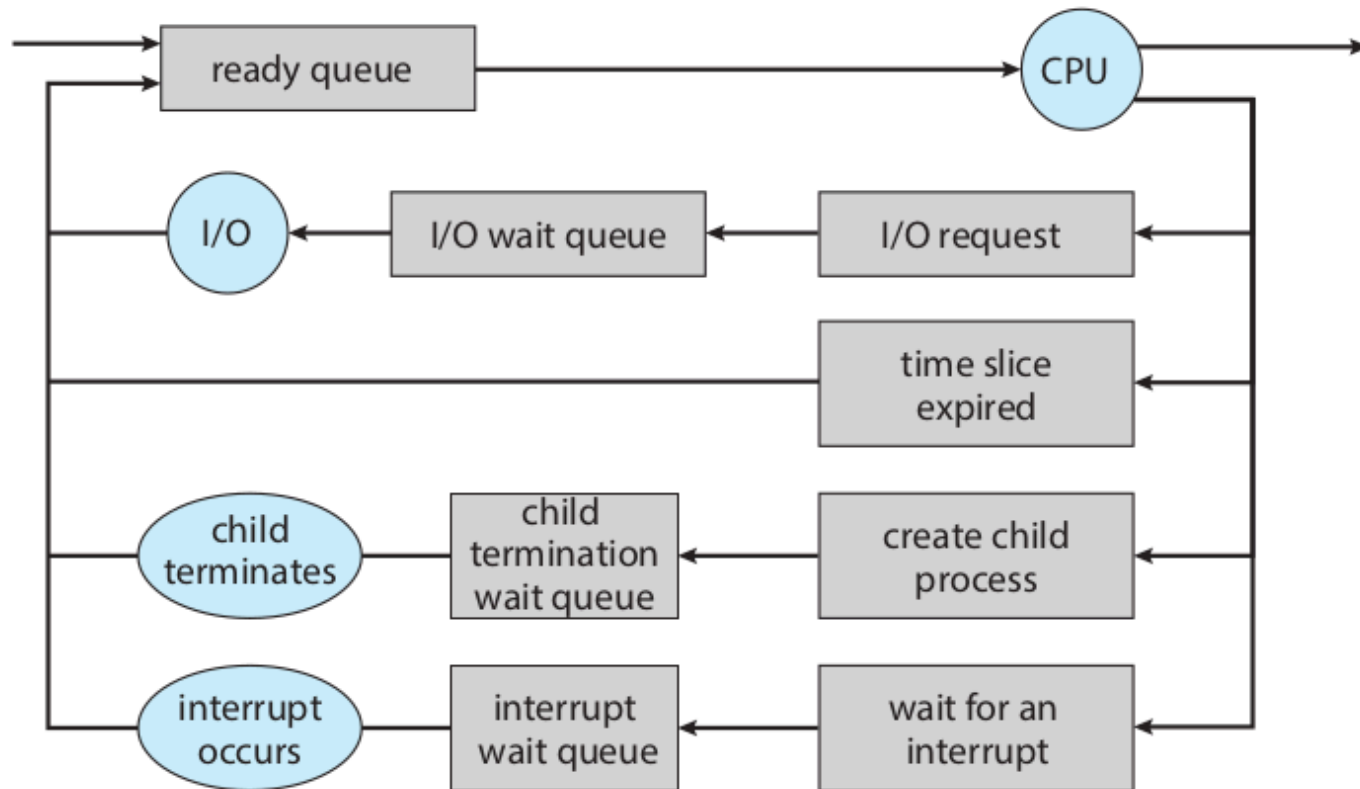
- **Processo I/O bound:** impiega la maggior parte del proprio tempo nell'esecuzione di operazioni di I/O
- **Processo CPU bound:** impiega la maggior parte del proprio tempo nelle elaborazioni

Il compito di uno scheduler è di **ottimizzare** l'esecuzione dei processi per farli eseguire nel minor tempo possibile

- Interviene più volte al secondo
- Gestisce l'esecuzione col meccanismo del **time sharing**

Scheduling

Ogni processo inizia dalla **Ready Queue** e segue il **diagramma di accodamento** finchè non termina



Scheduling

Lo scheduler decide a quale processa assegnare la/le CPU
Quando decide che (una) CPU deve essere assegnata a un altro processo, esso deve:

- Salvare lo stato del processo corrente
 - Per poterlo poi ripristinare quando il processo stesso potrà ritornare in esecuzione
- Caricare un nuovo processo ripristinandone lo stato salvato precedentemente

Si esegue un salvataggio dello stato e, in seguito, un corrispondente ripristino dello stato, detto **Context Switching**

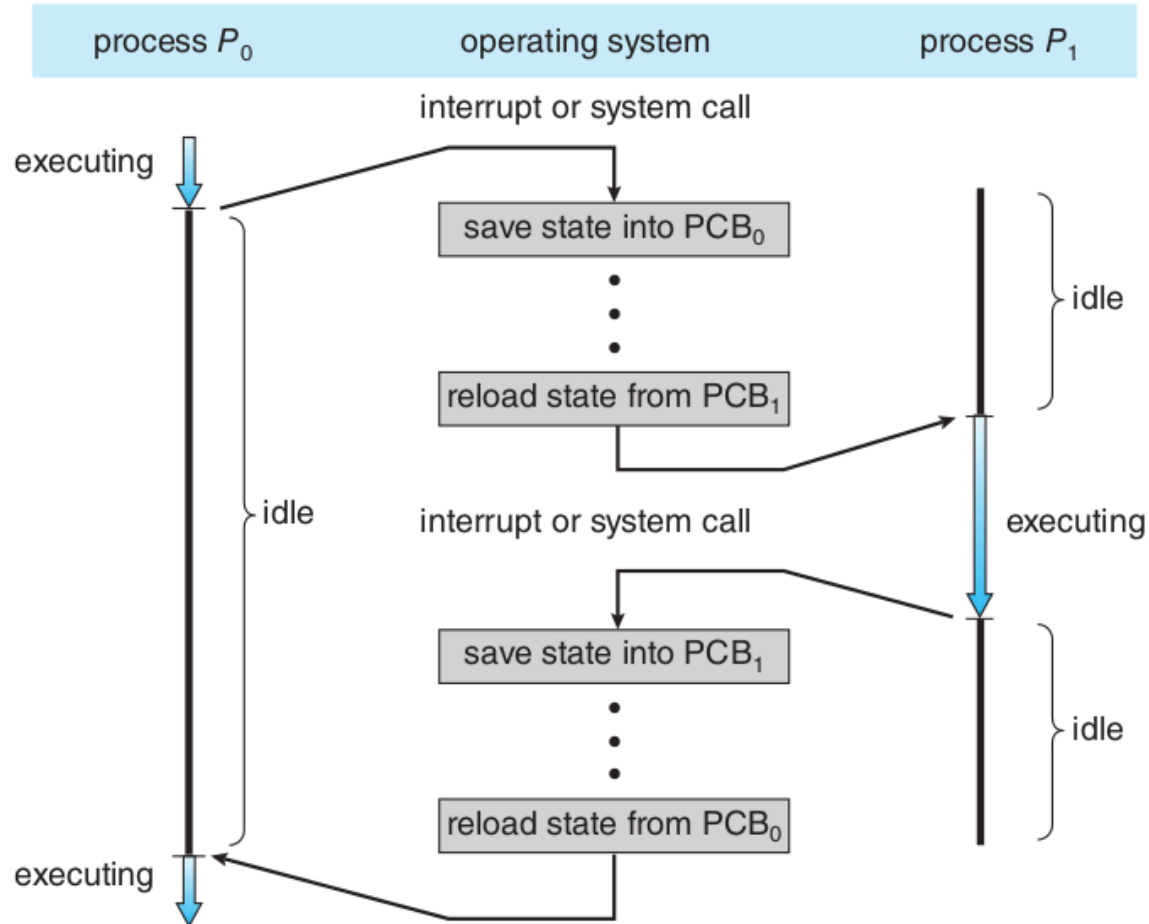
Scheduling

Il Context Switching deve essere rapido, siccome è tempo sprecato, che non svolge nessun compito utile

- Il SO è ottimizzato per compiere questa azione velocemente
- Attualmente nell'ordine di pochi micro secondi
- Dipende da hardware e dalle caratteristiche del processo, specialmente la quantità di memoria usata

Scheduling

Diagramma di cambio di contesto:



Scheduling

Operazione di Yield: un processo dice al kernel, che per il momento non ha operazioni da fare.

Il kernel rimuove dalla CPU il processo e lo riaccoda nella lista dei processi pronti.

- E' un modo per rilasciare la CPU prima che scada il quanto di tempo assegnato
- Specialmente usato per processi *real time*

In Linux:

```
#include <sched.h>
int sched_yield(void);
```


Algoritmi di Scheduling

Algoritmi di Scheduling

L'obiettivo di un SO è di ridurre il tempo di esecuzione dei processi

- Obiettivo ambiguo: conviene eseguire prima un lavoro lungo o uno corto?
- Obiettivo complesso: il SO non sa se un processo è lungo/corto, *CPU/I/O intensive*

Esistono diversi **Algoritmi di Scheduling** che si usano per determinare quale processo assegnare a una CPU

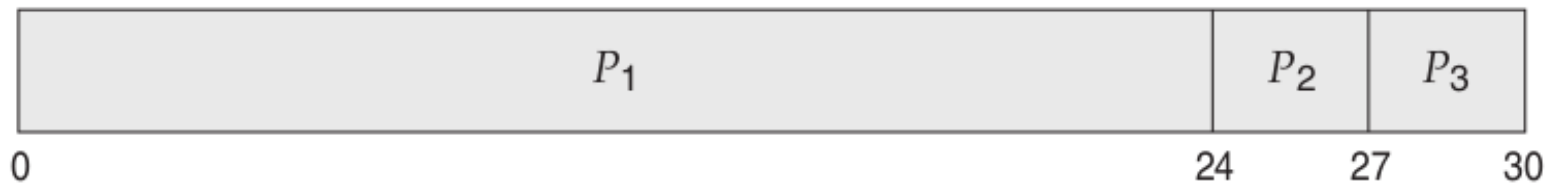
Algoritmi di Scheduling

First-Come First Served

Il primo processo che richiede la CPU, la ottiene finchè non termina

Pro Semplice

Contro: Inefficiente. Sconveniente eseguire un processo lungo per primo



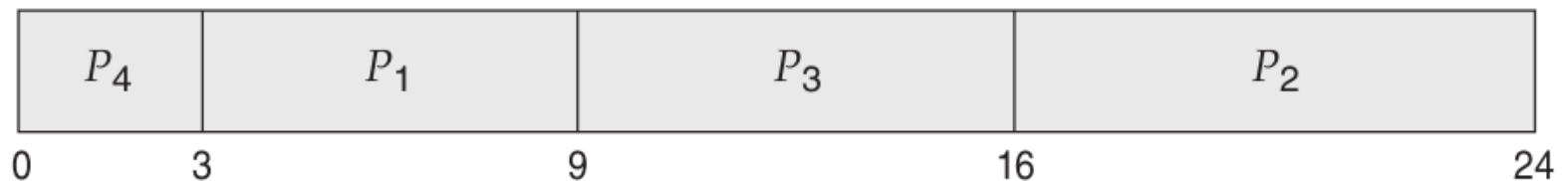
Algoritmi di Scheduling

Shortest-Job First Served

Il primo più breve, la ottiene la CPU per primo

Pro efficiente. Il tempo medio di completamento si abbassa

Contro: Inattuabile: non si sa quanto dura un processo



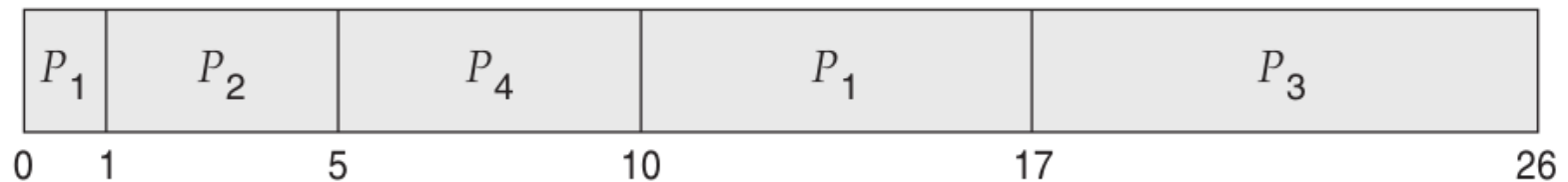
Algoritmi di Scheduling

Round Robin

A turno, ogni processo prende la CPU per un tempo fissato

Pro Semplice ed equo

Contro: Non si possono avere processi ad alta priorità



Algoritmi di Scheduling

Priority Scheduling

Ogni processo ha una priorità data dall'utente. Viene eseguito il processo a priorità più alta

Pro Gestisce la priorità

Contro: Un processo a bassa priorità potrebbe non venire mai eseguito

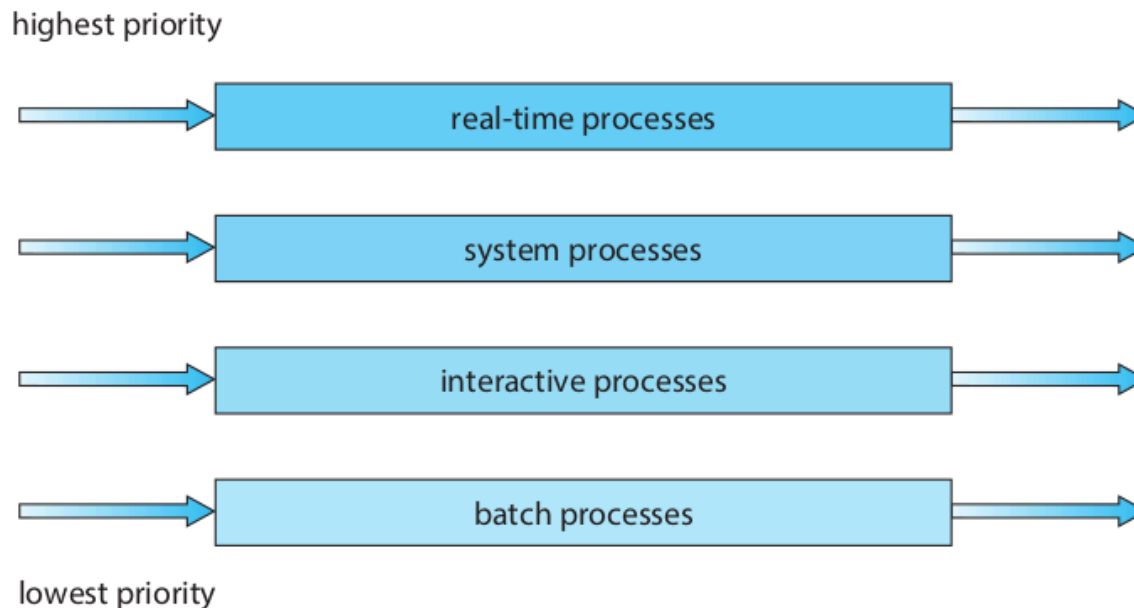


Algoritmi di Scheduling

Multi-Level Queue Scheduling

Ci sono code diverse per ogni livello livello di priorità.

- Ogni coda ha un suo algoritmo di scheduling: RR, FCFS
- C'è un algoritmo di scheduling tra code



Algoritmi di Scheduling

Multi-Level Queue Scheduling

In questo modo c'è flessibilità: si può avere priorità ma non c'è il rischio che un processo non venga mai eseguito

Pro Flessibile

Contro: Complesso

Usato in Linux, con varianti

Algoritmi di Scheduling

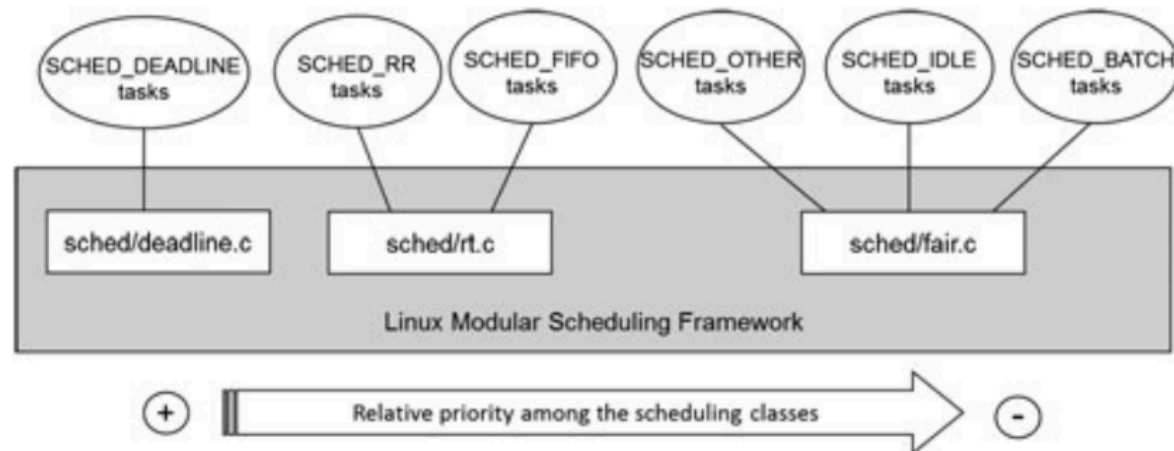
Linux: Completely Fair Scheduler

In Linux lo scheduler si chiama **Completely Fair Scheduler**

I processi sono assegnati a una **Policy di Scheduling** dall'utente, ognuna con meccanismi diversi

Il sistema provvede a eseguire processi in ogni policy, che hanno diversi requisiti

Le policy al loro interno possono gestire priorità, deadline, ecc...



Algoritmi di Scheduling

Linux: Completely Fair Scheduler

La Policy di default è `SCHED_OTHER` : time-sharing scheduling con priorità

Esistono System Call per definire priorità a un processo

```
nice(2) getpriority(2) setpriority(2)
```

```
sched_setscheduler(2) sched_getscheduler(2)
```

```
sched_setparam(2) sched_getparam(2) sched_yield(2)
```

Domande

Un processo è

- Una componente del sistema operativo
- Il codice eseguibile di un programma
- Un programma in esecuzione

I processi sono identificati da:

- Un nome
- Un ID numerico
- Non hanno identificativi

I processi sono in relazione tra loro in una struttura:

- Ciclica
- Ad albero

Il Context Switching è:

- La momentanea sospensione di un processo
- La terminazione di un processo

Nello scheduler di Linux, processi:

- Hanno tutti la stessa priorità di scheduling
- Hanno una priorità assegnabile dall'utente
- Hanno una priorità calcolata automaticamente dal SO

Sistemi Operativi
Unità 5: I processi
Operazioni sui processi

Martino Trevisan
Università di Trieste
Dipartimento di Ingegneria e Architettura

Argomenti

1. Creazione di un processo
2. Funzione `fork`
3. Funzione `wait`
4. Funzione `exec`
5. Funzione `system`
6. Funzione `exit`
7. Altre funzioni
8. Comandi Bash per Processi

Creazione di un processo

Creazione di un processo

In un SO, la manipolazione dei processi è effettuata tramite System Call

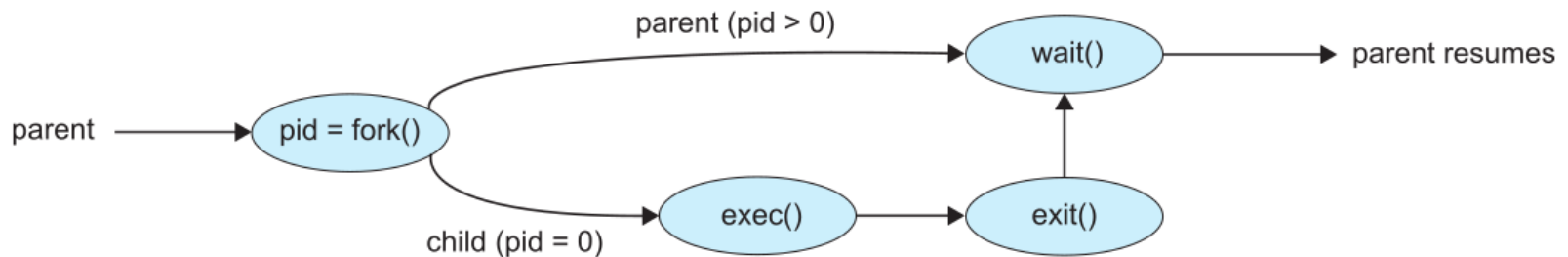
In Windows:

```
BOOL CreateProcess(  
    LPCSTR                lpApplicationName,  
    LPSTR                 lpCommandLine,  
    LPSECURITY_ATTRIBUTES lpProcessAttributes,  
    LPSECURITY_ATTRIBUTES lpThreadAttributes,  
    BOOL                  bInheritHandles,  
    DWORD                 dwCreationFlags,  
    LPVOID                lpEnvironment,  
    LPCSTR                 lpCurrentDirectory,  
    LPSTARTUPINFOA        lpStartupInfo,  
    LPPROCESS_INFORMATION lpProcessInformation  
);
```

Creazione di un processo

In Linux, esistono 6 System Call principali

- `fork` : crea un processo duplicato
- `exec` : carica un codice eseguibile
- `wait` : aspetta la terminazione di un figlio
- `kill` : invia un segnale
- `signal` : cattura un segnale
- `exit` : termina il processo corrente



Creazione di un processo

Windows vs Linux:

Windows ha una System Call complessa (`CreateProcessA`)

- Molto verboso
- Molti parametri
- Molto tipizzata

Linux preferisce System Call semplici:

- `fork` clona un processo
- `exec` permette di eseguire un file eseguibile nel processo corrente

Funzione `fork`

Funzione `fork`

```
#include <unistd.h>
pid_t fork (void);
```

Crea un nuovo processo figlio, copiando completamente l'immagine di memoria del processo padre (data, heap, stack)

- I due processi evolvono indipendentemente
- La memoria è completamente indipendente tra padre e figlio
- Il codice viene generalmente condiviso tra padre e figlio
 - Codice copy-on-write (copiato quando viene modificato)

Nota: `pid_t` è un alias per un `int`, come `size_t`

Funzione **fork**

```
#include <unistd.h>
pid_t fork (void);
```

- Tutti i descrittori dei file aperti nel processo padre sono duplicati nel processo figlio
- Sia il processo child che il processo parent continuano ad eseguire **l'istruzione successiva alla fork**
- Valore di ritorno:
 - Processo figlio: 0
 - Processo padre: PID del processo figlio
 - Errore della fork: PID negativo (solo padre)

Funzione **fork**

Esempio di utilizzo:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t  pid;
    pid = fork();
    if (pid == 0){
        printf("Sono il figlio!\n");
    }
    else{
        printf("Sono il Padre!\n");
    }
    return 0;
}
```

Funzione fork

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void Figlio(void);
void Padre(void);
int main()
{
    pid_t pid;
    pid = fork();
    if (pid == 0)
        Figlio();
    else
        Padre();
}
void Figlio(void)
{
    int i=0;
    for(i=0;i<10;i++){
        usleep(200);
        printf("\tSono il figlio. i= %d\n",i);
    }
}
void Padre(void)
{
    int i=1;
    for(i=0;i<10;i++){
        usleep(250);
        printf("Sono il padre. i= %d\n",i);
    }
}
```

Funzione `fork`

Osservazioni:

- Il valore di ritorno della `fork` è fondamentale
- Un programma scritto in termini di `fork` non è immediatamente comprensibile
 - Operazione atomica, ma effetti complessi
 - E' possibile creare alberi di processi complessi, con codice complesso

Funzione `fork`

Fork Bomb: un programma che chiama la `fork` in un ciclo infinito, blocca la macchina a causa dei troppi processi

```
#include <unistd.h>
int main(void)
{
    while(1)
        fork();
}
```


Funzione **fork**

Fork Bomb in Bash:

```
:(){ :|:& };:
```

che equivale a:

```
myfork() {  
    myfork | myfork &  
}  
myfork
```

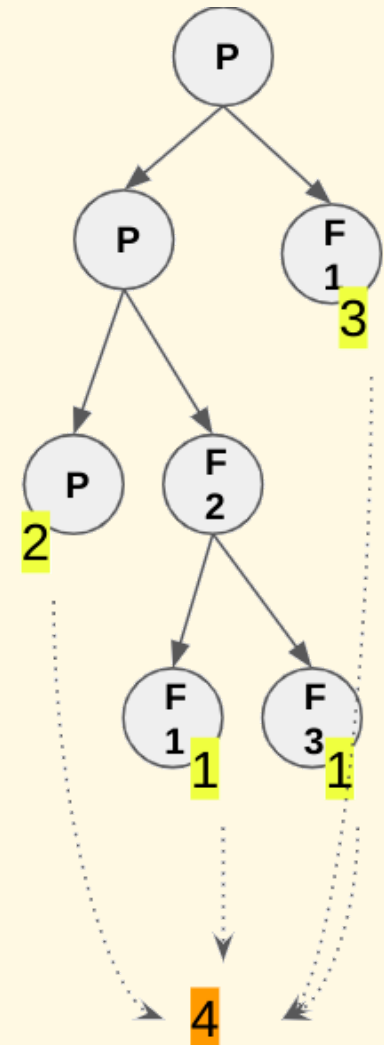
Funzione fork

Esercizio: si determini l'albero di processi generato dal seguente codice e l'output generato

```
#include <stdio.h>
#include <unistd.h>
int main(){
    if (fork()){
        if (!fork()){
            fork();
            printf("1 ");
        }
        else
            printf("2 ");
    }
    else
        printf("3 ");
    printf("4 ");
    return 0;
}
```

Output:

2 4 3 4 1 4 1 4



Funzione fork

Esercizio: si determini l'albero di processi generato dal seguente codice e l'output generato

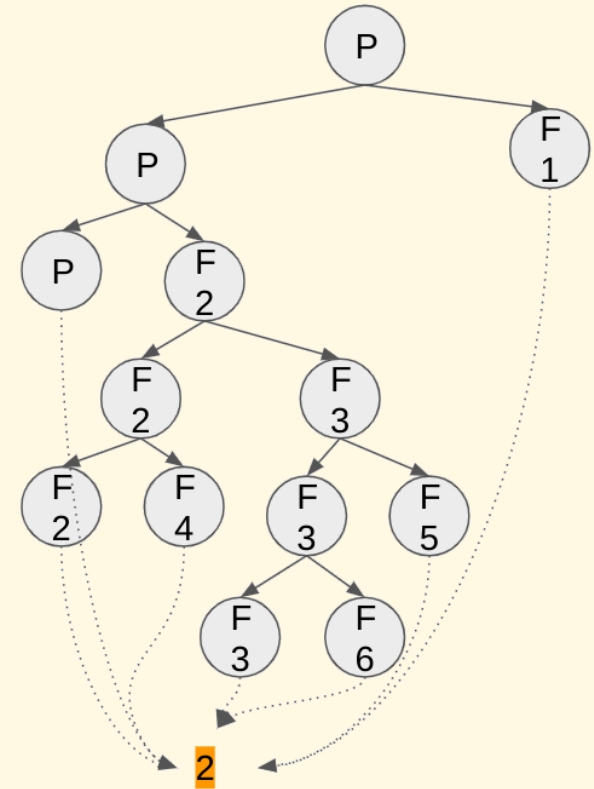
```
#include <stdio.h>
#include <unistd.h>

int main(){
    printf("\n");
    if (fork() && (!fork())) {
        if (fork() || fork()) {
            fork();
        }
    }

    printf("2 ");
    return 0;
}
```

Output:

2 2 2 2



Funzione `fork`

Esercizio: si determini l'output generato dal seguente programma

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char *argv[]){
    printf("A\n");
    fork();
    printf("B\n");
    fork();
    printf("C\n");
    return 0;
}
```

Output:

```
A
B
B
C
C
C
C
```

Funzione `fork`

Esercizio: si determini l'output generato dal seguente programma

Nota: non ci sono i `\n` nelle `printf`

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char *argv[]){
    printf("A ");
    fork();
    printf("B ");
    fork();
    printf("C ");
    return 0;
}
```

Output:

```
A B C A B C A B C A B C
```

Perchè?

Dipende dalla duplicazione della memoria dopo la `fork` e dall'I/O bufferizzato della `printf`

Funzione `wait`

Funzione `wait`

```
#include <sys/wait.h>
pid_t wait (int *status);
```

Attende la prima terminazione di **un** figlio

Argomento `status` :

- Puntatore ad un intero;
- Se non è NULL specifica lo stato di uscita del processo figlio (valore restituito dal figlio)

Valore di ritorno:

- Il PID del figlio terminato
- **0** in caso di errore

Funzione `wait`

Casistica:

- Se il processo non ha figli: Errore
- Se il processo ha dei figli che sono già terminati: ritorna istantaneamente
- Se il processo ha dei figli non ancora terminati: blocca il chiamante finchè non termina un figlio

Funzione `wait`

La funzione `wait` *consuma* un figlio per volta.

Dopo che un figlio è stato *ritornato* al padre tramite una `wait`:

- Il SO rilascia le risorse del processo figlio
 - Il SO mantiene informazioni su processi terminati di cui non è ancora stata effettuata una `wait`
 - Traccia che il processo è esistito
 - Valore di ritorno e informazioni su esecuzione
- Non verrà ritornato in successive invocazioni

Funzione `wait`

Processi Zombie: processo terminato il cui padre non ha ancora effettuato una `wait`

- Dopo che viene effettuata, il processo è morto definitivamente e non ne rimane traccia

Processi Orfani: processi in cui padre è morto.

- Se il padre muore, i figli continuano l'esecuzione
- Diventano figli del processo *init* (*PID* = 1)
- Periodicamente, *init* esegue delle `wait` per consumare gli orfani morti

Funzione `wait`

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);
```

Attende la prima terminazione di:

- Un qualsiasi figlio se `pid == -1` (come `wait` classica)
- Un figlio con PID `pid` se `pid > 0`
- Se `pid <= 0` altro comportamento (non coperto nel corso)

Valore di ritorno: `pid` del figlio

Funzione `wait`

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);
```

Altri argomenti di `waitpid` :

- `status` come nella `wait`
- `options` : controlla se la funzione è bloccante. E' una bitmask.
 - `0` bloccante
 - `WNOHANG` : non blocca in caso di assenza di figlio già morto
 - Altri flag per intercettare solo figli morti in condizioni particolari

Funzione `wait`

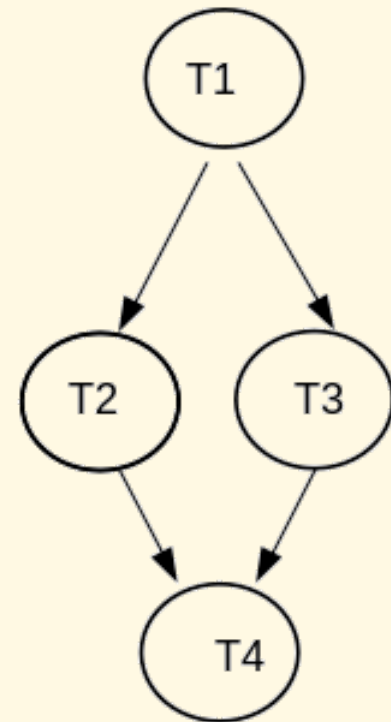
Esercizio: si scriva un programma che implementa il seguente grafo di precedenza con `fork` e `wait`.

Nota:

Ogni biforcazione si implementa tramite una `fork` e ogni ricongiungimento tramite una `wait`

Importante: questi esercizi permettono di scrivere codice che parallelizza diverse operazioni

Fondamentale per programmazione parallela



Funzione `wait`

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    printf ("T1\n");
    pid = fork();
    if (pid == 0) {
        printf ("T3\n");
        return 0;
    } else {
        printf ("T2\n");
        wait ((int *) 0);
    }
    printf ("T4\n");
    return 0;
}
```

Funzione `wait`

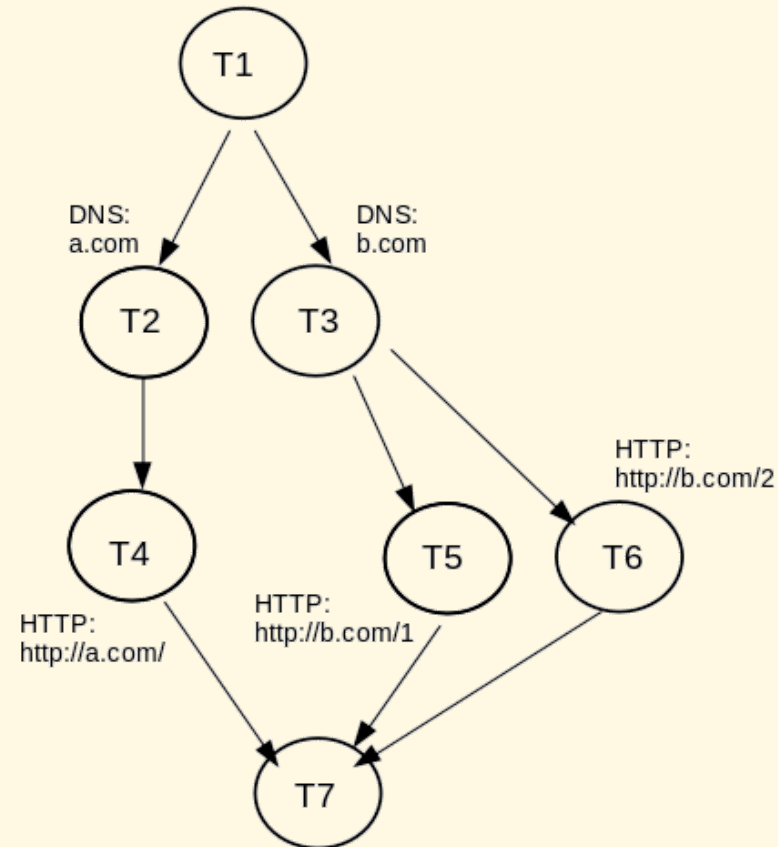
Esercizio: si scriva un programma che implementa il seguente grafo di precedenza con `fork` e `wait`.

Nota: questo è il grafo per eseguire in maniera efficiente 3 richieste HTTP alle URL.

- `http://a.com/`
- `http://b.com/1`
- `http://b.com/2`

Prima di ogni richiesta, è necessario effettuare la risoluzione DNS. Due URL hanno lo stesso dominio.

Nota: nei casi reali, il programmatore deve risolvere il problema efficientemente. Deve costruire da solo il grafo di precedenza.



Funzione `wait`

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    printf ("T1 - Start\n");
    pid = fork();
    if (pid == 0) {
        printf ("T3 - DNS b.com\n");
        pid_t pid2;
        pid2=fork();
        if (pid2==0){
            printf ("T6 - HTTP http://b.com/1\n");
            return 0;
        }
        else{
            printf ("T5 - HTTP http://b.com/2\n");
            wait ((int *) 0); /* Attende T6 */
            return 0;
        }
    } else {
        printf ("T2 - DNS a.com\n");
        printf ("T4 - HTTP http://a.com/\n");
        wait ((int *) 0); /* Attende T3 - T5 */
    }
    printf ("T7 - Utilizzo i risultati\n");
    return 0;
}
```

Nota: T5 aspetta T6 che è suo figlio. T7 non può *aspettare* T6, in quanto non è suo figlio

La `wait` *aspetta* solo sui figli, **NON** sui nipoti

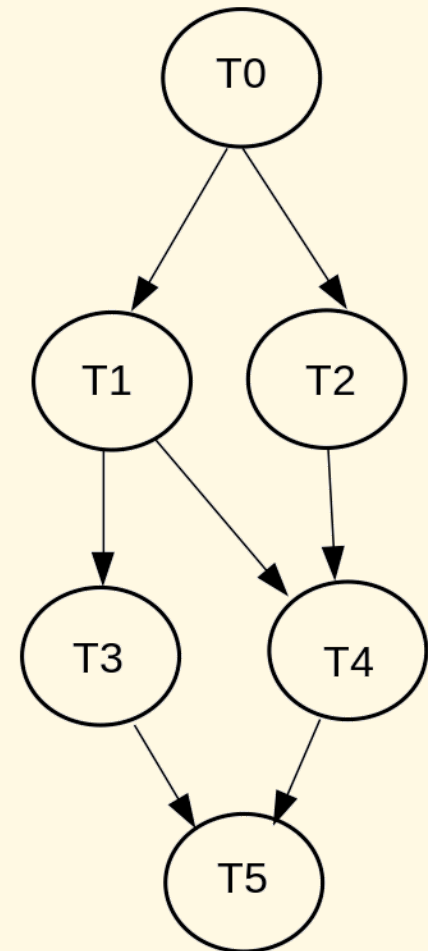
Funzione `wait`

Esercizio: si scriva un programma che implementa il seguente grafo di precedenza con `fork` e `wait`.

Questo grafo è **molto difficile** da realizzare mediante sole `fork` e `wait`

T4 non può *attendere* T1. Non è suo figlio!
In generale:

- Si possono *attendere* solo i figli
- Ogni figlio può essere atteso una volta sola



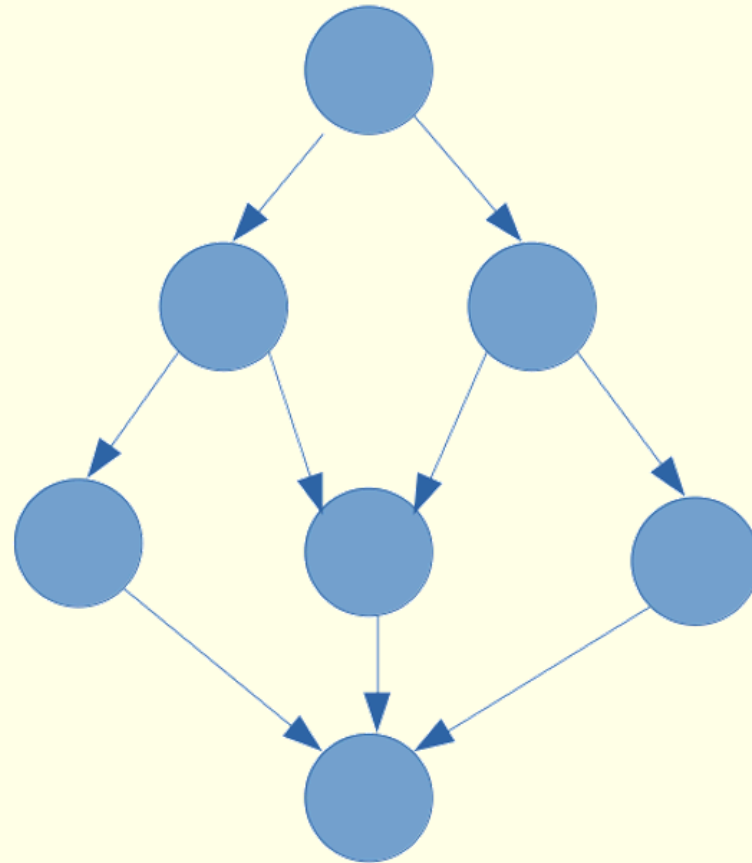
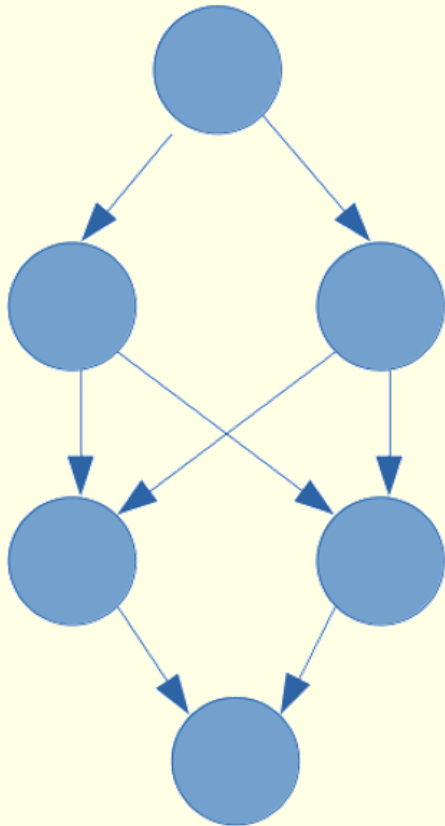
Funzione `wait`

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    printf ("T0\n");
    pid = fork();
    if (pid == 0) {
        printf ("T2\n");
        wait ( ??? ) /* <----- IMPOSSIBILE! */
        printf ("T4\n");
    } else {
        printf ("T1 -\n");
        printf ("T3 -\n");
        wait ((int *) 0); /* Attende T4 */
    }
    printf ("T7 - Utilizzo i risultati\n");
    return 0;
}
```

Come si può implementare questo grafo?

Funzione `wait`

Grafi impossibili:



Funzione **exec**

Funzione `exec`

La `fork` permette di duplicare un processo

- Usata quando il figlio deve eseguire lo stesso programma del padre
- In programmi paralleli: web server, database

La `exec` permette di cambiare la natura di un processo corrente

- Caricando ed eseguendo un programma diverso
- Usata ogniqualvolta bisogna avviare un nuovo programma

Funzione `exec`

Quando un processo chiama una `exec` :

- Il processo viene rimpiazzato **completamente** dal codice contenuto nel file specificato (text, data, heap, stack vengono sostituiti)
- Il nuovo programma inizia a partire dalla sua funzione `main`
- Il PID non cambia

Funzione `exec`

Cosa mantiene il processo dopo una `exec` :

- **Variabili d'ambiente**: le variabili definite nel terminale
 - Accessibili tramite la `char *getenv(const char *name)`
- PID e PPID (PID del padre)
- Privilegi, current working directory, root e home directory

Cosa non viene ereditato:

- File aperti se hanno il flag `close-on-exec`
- Altrimenti lasciati aperti

Funzione `exec`

Esistono 7 versioni della `exec`.

Hanno la stessa funzione, varia il modo in cui ricevono gli argomenti.

```
#include <unistd.h>

int execl(const char *pathname, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execlx(const char *pathname, const char *arg, ..., char *const envp[]);
int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(char *pathname, char *argv[], char* envp[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```


Funzione `exec`

Le funzioni con `p` ricevono il nome dell'eseguibile e non il path.

- Il SO rintraccia l'eseguibile nelle cartelle dei programmi installati nel sistema
- Che sono definite nella variabile d'ambiente `PATH`

Esempio: si equivalgono

```
execvp("cp", ...);  
execv("/usr/bin/cp", ...);
```

Perchè, sul mio PC:

```
$ echo $PATH  
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin
```

Funzione `exec`

- Le funzioni con `l` specificano gli **argomenti** del nuovo programma tramite una lista di argomenti. Simile a `printf`.
 - **Esempio:** `execlp("cp", "cp", "file1", "file2");`
Non dimenticare `argv[0]` !
- Le funzioni con `v` specificano gli **argomenti** del nuovo programma tramite un unico vettore di puntatori a `char`.
Equivalente a `argv` nel `main`
 - Il primo argomento deve contenere il nome del file associato all'eseguibile che viene caricato (`argv[0]`)
 - L'array di puntatori deve essere terminato da un puntatore NULL
 - Mancando `argc`, questo serve a comunicare la lunghezza del vettore

Funzione `exec`

Esempio:

```
// Semplice
execvp("cp", "cp", "file1", "file2");

// Generico
const char *args[4];
args[0] = "cp";
args[1] = "file1";
args[2] = "file2";
args[3] = NULL;
execvp("cp", args);
```

Funzione `exec`

Le funzioni con `e` ricevono un vettore di variabili d'ambiente. Quindi esse **non** vengono eridate dal processo esistente.

- Le variabili d'ambiente sono specificate nell'ultimo argomento tramite un vettore di puntatori a `char`
 - Terminato da puntatore `NULL`
 - Ogni elemento è una stringa nella forma `nome=valore`

```
char *const args[] = {"ls", "/tmp", NULL};  
execv("/usr/bin/ls", args);
```

```
char *const envs[] = {"a=1", "b=2", NULL};  
execve("/usr/bin/ls", args, envs);
```

Funzione `exec`

Osservazione:

- La funzione `execve` è una System Call.
- Le altre funzioni sono di libreria, e invocano la `execve` dopo aver correttamente gestito e aggiustato i parametri

Funzione `exec`

Esercizio: si scriva una semplice shell usando le funzioni `fork`, `wait` e `exec`

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#define MAXLINE 128
int main() {
    char    buf[MAXLINE];
    pid_t   pid;
    int     status;
    printf("%% "); /* prompt */
    while (fgets(buf, MAXLINE, stdin) != NULL) {
        if (buf[strlen(buf) - 1] == '\n')
            buf[strlen(buf) - 1] = 0;
        if ((pid = fork()) < 0) {
            printf("errore di fork "); exit(1);
        } else if (pid == 0) { /*figlio */
            execlp(buf, buf, NULL);
            printf("non posso eseguire: %s\n", buf);
            exit(127);
        } else
            if ((pid = waitpid(pid, &status, 0)) < 0) /* padre */
                {printf("errore di waitpid"); exit(1);}
            printf("%% ");
    }
    exit(0);
}
```

Nota: per gestire gli argomenti dei comandi invocati, bisognerebbe manipolare le stringhe

Funzione `exec`

Esercizio: si consideri il seguente programma.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main (int argc, char ** argv) {
    char str[10];
    int n;
    n = atoi(argv[1]) - 1;
    printf ("%d\n", n);
    if (n>0) {
        sprintf (str, "%d", n);
        execl (argv[0], argv[0], str, NULL);
    }
    printf ("End!\n");
    return 1;
}
```

Funzione `exec`

Cosa viene stampato eseguendo `./prog 5` ?

```
4
3
2
1
0
End!
```


Funzione **system**

Funzione `system`

E' una funzione di libreria che invoca un comando Bash e ne attende la conclusione

- Utile per usare programmi esterni in un programma
- Internamente usa: `fork`, `exec` e `wait`

```
#include <stdlib.h>

int system(const char *command);
```

Equivale a una `fork` il cui figlio esegue:

```
execl("/bin/sh", "sh", "-c", command, (char *) NULL);
```

Valore di ritorno: il valore di ritorno del comando eseguito

Funzione `system`

Implementazione semplice:

```
int system(const char *cmd)
{
    int stat;
    pid_t pid;
    if (cmd == NULL)
        return(1);
    if ((pid = fork()) == 0) { /* Son */
        execl("/bin/sh", "sh", "-c", cmd, (char *)0);
        _exit(127);
    }
    if (pid == -1) {
        stat = -1; /* Error */
    } else { /* Father */
        while (waitpid(pid, &stat, 0) == -1) {
            if (errno != EINTR){
                stat = -1;
                break;
            }
        }
    }
    return(stat);
}
```

Funzione `system`

Esercizio: Si scriva un programma che fa il listing dettagliato di una cartella.

- La cartella è passata come argomento
 - Se non ci sono argomenti, lista la directory corrente
- Usando `ls -lh cartella`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main (int argc, char * argv[1]) {
    char command[50] = "ls -lh ";

    if (argc == 2)
        strcat(command, argv[1]);

    system(command);

    return(0);
}
```

Funzione `exit`

Funzione `exit`

Ci sono diversi modi per terminare un processo:

1. Modo Standard

- Dal `main` avviene una `return`

```
return status;
```

- Viene chiamata la funzione `exit`

```
#include <stdlib.h>  
void exit(int status);
```

Tutti i buffer (di console e file) vengono *flushed*

L'argomento `status` è ritornato al SO

Funzione `exit`

Per permettere queste operazioni di pulizia, vengono chiamate tutte le funzioni di chiusura:

- Della libreria standard
- Definite dall'utente tramite la funzione:

```
int atexit(void (*function)(void));
```

```
void fun(void) { printf("Exiting\n"); }

int main()
{
    atexit(fun);
    exit(10);
}
```

Viene stampato `Exiting`

Funzione `exit`

2. System Call `_exit`

```
#include <unistd.h>
void _exit(int status);
```

Termina immediatamente senza invocare nessuna funzione di pulizia.

Invocata nei processi **figli** che potrebbero leggere *buffer* in stato intermedio dei padri

Usata specialmente dopo `exec` fallite

- Il figlio non dovrebbe eseguire nessuna istruzione dopo la `exec` !
- I buffer possono contenere dati del padre che non devono essere scritti dai figli

Funzione `exit`

Nota:

- La `_exit` è una System Call
- La `exit` è una funzione di libreria. Fa pulizia e poi invoca la `_exit`

3. Terminazione Anomala:

- Viene ricevuto un **segnale** non gestito (vedremo)
- Il programma chiama la `abort`.

```
#include <stdlib.h>
void abort(void);
```

Funzione `exit`

In qualunque modo termini il processo, il kernel compie le seguenti azioni:

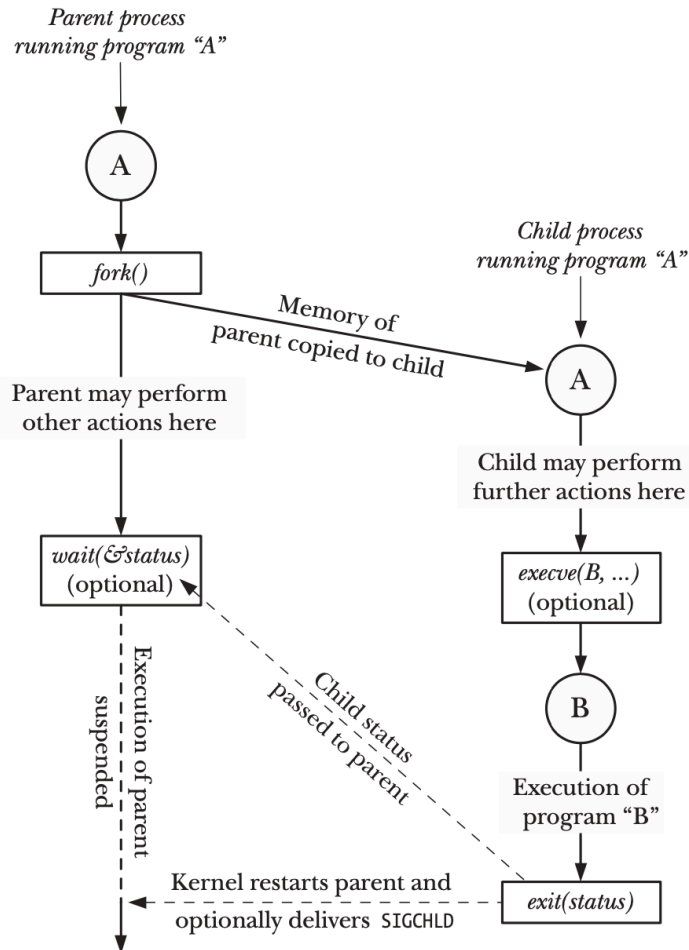
- Rimozione della memoria utilizzata dal processo
- Chiusura dei descrittori aperti

Stato di un processo: raccolto con `wait()`, `waitpid()`

Funzione `exit`

Per riassumere

- `fork` duplica il processo corrente
- `execve` tramuta il processo corrente in un altro programma
- `exit` termina il processo corrente (uguale a `return` dal `main`)
- `wait` blocca finchè un processo figlio non termina



Altre funzioni

Altre funzioni

```
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

La `getpid()` ritorna il PID del processo chiamante

La `getppid()` ritorna il PID del **padre** del processo chiamante

Comandi Bash per Processi

Comandi Bash per Processi

- `ps` : lista i processi del sistema
 - Di default mostra solo processi figli del terminale corrente.
Con l'opzione `a` mostra tutto
 - Di default, mostra solo processi che sono in foreground (hanno una shell)
Con opzione `x` mostra anche quelli in background
 - Opzioni utili: `u` mostra utente proprietario. `f` rende graficamente gerarchia padre-figlio
- `top` : mostra i processi in maniera interattiva
- `htop` : come `top` ma grafica migliorata

Comandi Bash per Processi

- `which` : fornisce il path assoluto di un programma di sistema

```
$ which ls  
/usr/bin/ls
```

- I comandi di sistema vengono cercati nelle cartelle indicate nella variabile d'ambiente `$PATH`

- Solitamente:

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin
```

- `pgrep` : stampa il PID di tutti i processi di un programma

```
$ pgrep chrome  
480492  
480498  
480505
```


Comandi Bash per Processi

Esecuzione di processi figli da script bash:

- Un comando che termina con `&` viene eseguito in background
 - Viene eseguita una `fork` e una `exec` per eseguire il comando
 - Non esegue la `wait`. Lo script e il programma eseguono in parallelo
- Il PID del processo appena creato può essere ottenuto con `$!`
 - Sovrascritto a ogni processo creato!
- Si può usare il comando `wait [PID]`
Attende il figlio `PID` se specificato, altrimenti un figlio qualsiasi

Comandi Bash per Processi

Esempio: Quanto ci mette a eseguire questo codice?

```
sleep 4 &      # Sleep viene eseguito in background
PID=$!        # Lo script recupera il PID
sleep 2
wait $PID      # Lo script attende che sleep termini
```

Ci mette 4 secondi, non 6

Comandi Bash per Processi

II `/proc` file system

I sistemi Linux/POSIX espongono informazioni sui processi correnti tramite uno **Pseudo File System** detto `procfs`

- File System Virtuale
- Montato in `/proc` in automatico
- Permette a chiunque di conoscere lo stato dei processi in esecuzione
- Tramite normali letture da file

Comandi Bash per Processi

Il `/proc` file system

Le informazioni su un processo *PID* si trovano nella directory `/proc/PID`

Il file `/proc/PID/status` contiene varie informazioni:

```
$ cat /proc/1566/status
Name:      grep
State:     R (running)
Tgid:      1566
Pid:       1566
PPid:      743
...
VmPeak:    5004 kB
VmSize:    5004 kB
VmLck:     0 kB
VmHWM:     476 kB
VmRSS:     476 kB
```

Comandi Bash per Processi

Il `/proc` file system

La **subdirectory** `/proc/PID/fd` contiene un link per ogni file aperto dal processo

- Il nome di questi link è il numero del descrittore usato nel processo
- Ricordare: ogni file aperto identificato da un numero

Esempio:

```
/proc/1968/fd/1
```

Rappresenta lo `stdout` del processo **1968**.

- Ricorda: **0** è `stdin`, **1** è `stdout`, **3** è `stderr`

Comandi Bash per Processi

II `/proc` file system

Altri **file/subdirectory** del processo *PID* sotto `/proc/PID`

File	Description (process attribute)
cmdline	Command-line arguments delimited by \0
cwd	Symbolic link to current working directory
environ	Environment list <i>NAME=value</i> pairs, delimited by \0
exe	Symbolic link to file being executed
fd	Directory containing symbolic links to files opened by this process
maps	Memory mappings
mem	Process virtual memory (must <i>lseek()</i> to valid offset before I/O)
mounts	Mount points for this process
root	Symbolic link to root directory
status	Various information (e.g., process IDs, credentials, memory usage, signals)
task	Contains one subdirectory for each thread in process (Linux 2.6)

Comandi Bash per Processi

II `/proc` file system

Il file system `/proc` fornisce anche molte informazioni sul sistema e possibilità di configurazione

- `/proc/cpuinfo` : informazioni su CPU
- `/proc/meminfo` : informazioni su memoria

Directory	Information exposed by files in this directory
<code>/proc</code>	Various system information
<code>/proc/net</code>	Status information about networking and sockets
<code>/proc/sys/fs</code>	Settings related to file systems
<code>/proc/sys/kernel</code>	Various general kernel settings
<code>/proc/sys/net</code>	Networking and sockets settings
<code>/proc/sys/vm</code>	Memory-management settings
<code>/proc/sysvipc</code>	Information about System V IPC objects

Comandi Bash per Processi

Altri Pseudo File System

- `sysfs` : montato in `/sys` , contiene informazioni sullo **stato del kernel** e sulle periferiche
 - Complementare a `/proc`
- `/dev` : contiene i file speciali che rappresentano le **periferiche**
 - Dispositivi a blocchi:
 - Dischi: `/dev/sda1` , `/dev/hda2`
 - CDRom: `/dev/cdrom` ; Floppy: `/dev/fd0`
 - Dispositivi a carattere: tastiera, mouse
 - Quando si legge/scrive a questi file speciali, viene invocato il **driver** della periferica corrispondente

Domande

L'esecuzione del seguente codice quanti processi genera (incluso il processo che esegue il `main`) ?

```
#include <stdio.h>
#include <unistd.h>
int main(){
    int N = 2;
    for (i=0; i<N; i++)
        fork();
}
```

- 2
- 3
- 4
- 6

Un processo il cui processo padre muore:

- Viene terminato dal SO
- Riceve un segnale dal SO
- Viene ereditato (diventa figlio) dal processo `init`

Domande

Cosa stampa il seguente codice?

```
#include <stdio.h>
#include <unistd.h>
int main(){
    if ( fork() ){
        printf("A\n");
    }else{
        fork();
        printf("B\n");
    }
}
```

- A
B
B
- A
A
A
- A
B
- A
A
B

La System Call `execve` crea un nuovo processo?

- Sempre
- Mai
- Dipende da come viene invocata

La funzione `system` crea un nuovo processo?

- Sempre
- Mai
- Dipende da come viene invocata

Sistemi Operativi

Unità 5: I processi

I Segnali

Martino Trevisan
Università di Trieste
Dipartimento di Ingegneria e Architettura

Argomenti

1. Concetto di segnale
2. Segnali in Linux
3. System Call `sigaction`
4. System Call `kill`
5. System Call `raise`
6. System Call `pause`
7. System Call `alarm`
8. Considerazioni
9. Segnali nella shell

Concetto di segnale

In quasi tutti i sistemi ad elaboratore, esistono gli **interrupt**:

Un interrupt informa la CPU che deve interrompere il compito corrente per eseguire un'azione impellente

Un interrupt viene generato da:

- Un dispositivo hardware che vuole notificare al sistema un evento
- Particolari istruzioni nel codice (e.g., istruzione `INT`)
 - Quando un processo chiama una System Call genera un interrupt software

Concetto di segnale

Un **segnale** permette la gestione di eventi asincroni che interrompono il normale funzionamento di un processo

- Idea simile a Interrupt, ma nel mondo dei processi
- **Notifica** un evento a un processo specifico

Possono essere **generati** da

- Kernel per comunicare eventi eccezionali:
 - Condizioni di errore
 - Azioni dell'utente (e.g., `CTRL+C` su tastiera)
- Un altro processo (se ne ha i permessi):
 - Permettono una primitiva comunicazione tra processi
 - Usando la system call `kill`

Concetto di segnale

Esistono dalle prime versioni di **Unix**

- Formalizzati in Unix 4

In principio erano inaffidabile e gestiti in modo *best-effort*

- Potevano andare perduti
- La gestione era complicata
- Poca configurazione possibile

I segnali esistono anche in **Windows**, sebbene abbiano un funzionamento leggermente diverso

Segnali in Linux

Esistono diversi tipi di segnali in Linux

- Dipende dalle versioni di Linux
- Comando `kill -l` lista i segnali
 - 64 in Ubuntu 20

Un tipo di segnale ha un identificatore mnemonico e numerico

- Identificatori di segnali iniziano con i tre caratteri SIG
- Es. `SIGINT` è il segnale di interruzione e ha numero 2
 - Mapping contenuto in `signal.h`

I segnali **non** hanno payload

Segnali in Linux

Ogni segnale viene generato da un evento specifico nel SO, o *manualmente* da un processo.

Un segnale può avere i seguenti effetti su un processo:

Importanti

1. Viene ignorato
2. Termina il processo
3. Interrompe momentaneamente il processo. Esegue una **funzione handler**. Dopodichè il processo riprende

Secondari

4. Crea un **core dump**: un file che contiene lo stato del programma per poter essere debuggato
5. Stoppa il processo
6. Fa ripartire il processo

Segnali in Linux

Ogni segnale ha comportamento di default

Modificabile (tranne che in casi specifici)

Segnali che di default terminano il processo:

- `SIGINT` : inviato al processo quando si preme `CTRL+C`
- `SIGTERM` : Segnale di terminazione normalmente usato. Generato dal comando `kill` di default
- `SIGHUP` : Inviato ad un processo se il terminale viene disconnesso
- `SIGKILL` : Uccide un processo. **Comportamento non modificabile**
- `SIGUSR1` e `SIGUSR2` : generati solo da processi utente, mai dal SO. Servono per comunicazione tra processi

Segnali in Linux

Segnali che di default terminano il processo:

- `SIGABRT` : inviato da system call `abort()`
- `SIGFPE` : inviato da eccezione aritmetica
- `SIGSEGV` : Accesso di memoria non valido

Segnali ignorati di default:

- `SIGCHLD` : inviato al padre quando un figlio termina

Altro:

- `SIGSTOP` : mette in pausa il processo. **Comportamento non modificabile**
- `SIGCONT` : riavvia il processo.

Segnali in Linux

Lista più completa.

Il comportamento di default può essere modificato:

- Per ignorare un segnale
- Per *gestirlo* tramite un *handler*
- **NON** per indurre Terminazione o Core Dump
- Tranne `SIGKILL` e `SIGSTOP`

Name	Description	Default
SIGABRT	Abort process	Core
SIGALRM	Real-time timer expiration	Term
SIGBUS	Memory access error	Core
SIGCHLD	Child stopped or terminated	Ignore
SIGCONT	Continue if stopped	Cont
SIGFPE	Arithmetic exception	Core
SIGHUP	Hangup	Term
SIGILL	Illegal Instruction	Core
SIGINT	Interrupt from keyboard	Term
SIGIO	I/O Possible	Term
SIGKILL	Sure kill	Term
SIGPIPE	Broken pipe	Term
SIGPROF	Profiling timer expired	Term
SIGPWR	Power about to fail	Term
SIGQUIT	Terminal quit	Core
SIGSEGV	Invalid memory reference	Core
SIGSTKFLT	Stack fault on coprocessor	Term
SIGSTOP	Sure stop	Stop
SIGSYS	Invalid system call	Core
SIGTERM	Terminate process	Term
SIGTRAP	Trace/breakpoint trap	Core
SIGTSTP	Terminal stop	Stop
SIGTTIN	Terminal input from background	Stop
SIGTTOU	Terminal output from background	Stop
SIGURG	Urgent data on socket	Ignore
SIGUSR1	User-defined signal 1	Term
SIGUSR2	User-defined signal 2	Term
SIGVTALRM	Virtual timer expired	Term
SIGWINCH	Terminal window size changed	Ignore
SIGXCPU	CPU time limit exceeded	Core
SIGXFSZ	File size limit exceeded	Core

Segnali in Linux

Un processo può definire un **signal handler**.

- Una funzione che viene eseguita quando il processo riceve il segnale
- Se non lo fa, c'è il comportamento di default

"Se e quando ricevi un certo segnale, esegui questa funzione"

Segnali in Linux

Fasi di vita di un segnale:

1. **Generazione:** da parte del kernel o di un processo
2. **Consegna:** nel più breve tempo possibile consegna il segnale al processo.
 - Finchè un segnale non è consegnato è *pending*
3. **Gestione:**
 - Il kernel avvia la funzione handler del processo nel caso ce ne sia una
 - Altrimenti compie l'azione di default per quel segnale (termina o ignora)

Segnali in Linux

Osservazione:

I segnali non vengono accodati.

I segnali pendenti per un processo sono una *mask*

- Se lo stesso segnale è generato più volte prima che sia consegnato, esso lo sarà una volta sola

System Call **sigaction**

```
#include <signal.h>
int sigaction(int signum, const struct sigaction *act,
              struct sigaction *oldact);
```

Modifica il comportamento del processo corrente a un segnale particolare

Argomenti:

- **signum** : segnale da trattare
- **act** : puntatore a struttura che definisce trattamento
- **oldact** : puntatore a comportamento precedente. Può servire per ristabilire il comportamento precedente

Ritorna -1 se c'è stato errore

System Call `sigaction`

```
struct sigaction {  
    void      (* sa_handler )( int );  
    sigset_t   sa_mask ;  
    int        sa_flags ;  
    void      (* sa_restorer )( void );  
};
```

- `sa_handler` specifica il comportamento
 - Se funzione, specifica un handler
 - Se `SIG_IGN` ignora
 - Se `SIG_DFL` ripristina comportamento di default

System Call `sigaction`

```
struct sigaction {  
    void (* sa_handler )( int );  
    sigset_t sa_mask ;  
    int sa_flags ;  
    void (* sa_restorer )( void );  
};
```

- `sa_mask` : segnali da bloccare mentre l'handler è in esecuzione
 - Si inizializza con `int sigemptyset(sigset_t *set);`
 - Nel corso non faremo altre operazioni su questo campo
- `sa_flags` : flag (non vediamo). Settiamo sempre a **0**
- `sa_restorer` : per uso interno

System Call `sigaction`

Esempio: codice per ignorare il segnale `SIGINT`

```
int sig = SIGINT;
struct sigaction sa ;
sa.sa_handler = SIG_IGN ;
sa.sa_flags = 0;
sigemptyset (&sa.sa_mask );
if (sigaction ( sig , &sa , NULL ) == -1)
    perror("sigaction failed");
```

System Call **sigaction**

La funzione handler deve prendere un argomento `int`

- Quando invocata dal SO, contiene il numero del segnale

E ritornare `void`

```
void myHandler ( int sig )  
{  
    /* Actions to be performed when signal  
    is delivered */  
}
```

Nota: L'handler è una funzione del programma.

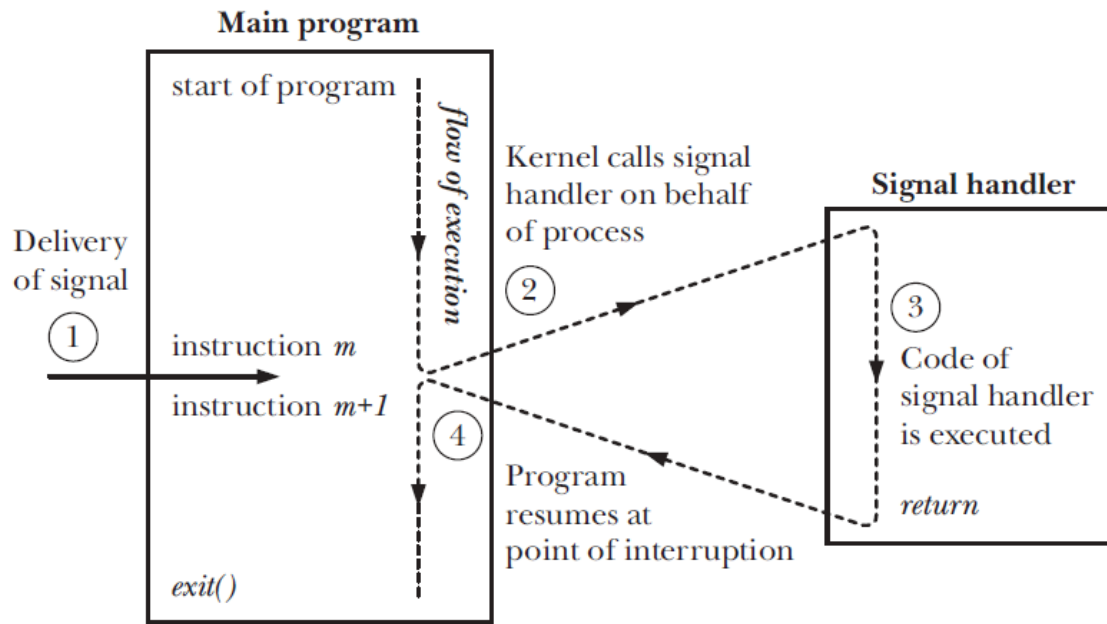
- Che viene invocata **automaticamente** dal SO alla ricezione del segnale
- E non dal programmatore. Potrebbe, ma non ha senso

System Call **sigaction**

Viene invocata automaticamente dal kernel alla consegna del segnale

Il programma si interrompe, esegue l'handler

Infine, continua l'esecuzione dal punto di interruzione



System Call `sigaction`

Esempio: si crei un programma che gestisce i segnali `SIGINT` e `SIGTERM`

```
#include <signal.h>
#include <stdio.h>
void func(int signum){
    printf("ricevo %d\n", signum);
}

int main (int argc, char *argv[]){
    struct sigaction new_action, old_action;

    new_action.sa_handler = func;
    sigemptyset ( &new_action.sa_mask); /* Si noti l'uso di sigemptyset */
    new_action.sa_flags = 0;

    sigaction (SIGINT, &new_action, NULL);
    sigaction (SIGTERM, &new_action, NULL);

    while(1) ;
    return 0;
}
```

Per terminare il programma, bisogna mandargli un segnale `SIGKILL`.

```
kill -KILL <nome prog>
```

System Call `sigaction`

Esiste la System Call `signal`, più a basso livello.

```
#include <signal.h>
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);
```

Argomenti:

- `sig`: quale segnale gestire
- `handler` specifica il comportamento. E' puntatore a funzione.

Nota: consigliato usare `sigaction`

System Call **kill**

Manda un segnale ad un processo oppure a un gruppo di processi

```
#include <sys/types.h>
#include <signal.h>
int kill(pid_t pid, int sig);
```

Argomenti:

- **sig** : segnale da mandare
- **pid** :
 - se **> 0** : spedito al processo identificato da **pid**
 - se **0** : spedito a tutti i processi appartenenti allo stesso gruppo del processo che invoca **kill()**
 - se **<0** : spedito al gruppo di processi identificati da **-pid**
 - se **-1** : a tutti i processi (attenzione!)

Privilegi: un processo può mandare un segnale solo a processi dello stesso utente. Tranne **root**, che può mandare a tutti.

System Call `sigaction`

Esercizio: si crei un programma che genera un processo figlio. Il padre manda al figlio un segnale `SIGUSR1` ogni secondo. Il figlio stampa l'avvenuta ricezione.

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void handler(int signum){
    printf("Ricevuto\n");
}

int main (int argc, char *argv[]){
    pid_t pid;
    struct sigaction action;

    pid = fork();
    if (pid!=0){ /* Father */
        while(1){
            sleep(1);
            kill (pid, SIGUSR1);
            /* pause(); Would be equivalent */
        }
    } else{ /* Child */
        action.sa_handler = handler;
        sigemptyset (&action.sa_mask);
        action.sa_flags = 0;
        sigaction (SIGUSR1, &action, NULL);
        while (1);
    }
    return 0;
}
```

System Call `raise`

```
#include <signal.h>
int raise (int sig);
```

Permette a un processo di inviare un segnale a se stesso.

Di fatto:

```
raise (sig);
```

equivale a:

```
kill (getpid(), sig);
```

System Call **pause**

```
#include <unistd.h>  
int pause (void);
```

Sospende il processo fino all'arrivo di un segnale

Serve a implementare l'attesa passiva di un segnale

Ritorna dopo che il segnale è stato catturato ed il gestore è stato eseguito, restituisce sempre (-1)

System Call `alarm`

```
#include <unistd.h>
unsigned int alarm (unsigned int seconds);
```

Implementa un timeout

Il SO manda un segnale `SIGALRM` al processo dopo `seconds` secondi

- Se non vi era già un timeout settato, restituisce `0`
- Altrimenti, restituisce i secondi che mancano allo scadere dell'ultimo allarme settato. Cancella il vecchio timeout e inserisce il nuovo
- Se `seconds` è `0`, si disattiva il timeout

System Call **alarm**

Osservazioni:

Il timeout è gestito dal kernel.

Il tempo effettivo può essere leggermente maggiore a causa del tempo di reazione del kernel

System Call `alarm`

Esempio: funzione `sleep` implementata con `alarm` e `pause`

```
static void myAlarm (int signo) {  
    return;  
}  
void mySleep (unsigned int nsecs) {  
    signal(SIGALRM, myAlarm)  
    alarm (nsecs);  
    pause ();  
}
```

Considerazioni

Un handler è un flusso di esecuzione concorrente

- Può iniziare in qualsiasi istante
- Mentre il flusso principale sta compiendo qualsiasi azione

Importante:

L'handler non deve modificare variabili globali che sono usate anche dal flusso principale

- Potrebbe portare in stato inconsistente

Considerazioni

- Immaginiamo che il programma venga interrotto tra la riga **2** e la riga **3** del seguente codice:

```
1 local = global; /* Supponiamo global = 1 */
2 local++;        /* Local = 2 */
--- Interruzione ---
3 global = local;
```

- L'handler esegue l'operazione:

```
global++;        /* Global = 2 */
```

- La variabile `global` viene incrementata
- Il programma riprende dall'istruzione **3**.

```
...
--- Interruzione ---
3 global = local; /* Global = 2 */
```

- L'incremento dell'handler si è perso. `global` = **2** anzichè **3**

Problema che vedremo approfonditamente per i programmi multi-thread

Considerazioni

Definizioni:

Funzione rientrante: può essere usata con sicurezza in più flussi

Funzione non rientrante: **NON** può essere usata con sicurezza in più flussi

In generale, negli handler, bisogna:

- Chiamare solo funzioni rientranti
- Evitare di manipolare variabili globali che sono usate dal flusso principale.

Considerazioni

- Molte funzioni di libreria C **NON** sono rientranti
 - `fprintf`, `fscanf`: gestione del **buffer** problematica.
Non vanno chiamate dentro un handler!
- Alcune funzioni sono rientranti e possono essere interrotte senza problemi: `sleep`, etc. Vedi `man`
- Le **System Call** sono rientranti. Ci pensa il kernel a gestire tutto

Segnali nella shell

Segnali nella shell

```
kill pid
```

Invia un segnale al processo `PID` .

Di default manda `SIGTERM` .

Possibile specificare con opzioni `-KILL` `-INT`

```
pkill nome  
killall nome
```

Stesso comportamento, ma manda il segnale a tutti i processi del programma `nome`

Segnali nella shell

Esercizio: si scriva un programma in C che memorizza quanti `SIGTERM` ha ricevuto. Alla pressione di `CTRL+C` stampa tale numero e termina. Si nomini il programma `sample`.

Si scriva anche uno script bash che manda 5 segnali `SIGTERM` al processo.

Programma Bash:

```
for i in $( seq 5) ; do  
    pkill sample  
done
```

Segnali nella shell

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int c;

void handler(int signum){
    if (signum==SIGTERM)
        c+=1;
    else if (signum==SIGINT){
        printf("Ricevuti %d SIGTERM\n", c);
        exit (0);
    }
}

int main (int argc, char *argv[]){
    struct sigaction action;
    c=0;
    action.sa_handler = handler;
    sigemptyset (&action.sa_mask);
    action.sa_flags = 0;
    sigaction (SIGTERM, &action, NULL);
    sigaction (SIGINT, &action, NULL);

    while (1);
    return 0;
}
```

Segnali nella shell

Quando si preme `CTRL+C`, viene mandato un `SIGINT` al programma, che stampa `c` e termina

Extra: si faccia uno script bash che automatizza tutta la sequenza: avvio del programma in C, consegna segnali e chiusura.

```
./sample &  
PID=$!  
for i in $( seq 5) ; do  
    kill $PID  
done  
kill -INT $PID
```

Segnali nella shell

Se `CTRL+C`, viene inviato `SIGINT`

- Programma termina se non c'è un handler

Se `CTRL+Z` viene inviato `SIGTSTP`

- Di default l'applicazione viene sospesa
 - Stesso effetto di `SIGSTOP`, ma per `SIGTSTP` si può definire handler
- E messa in background dalla shell
- A questo punto:
 - `fg` fa riprendere l'esecuzione in foreground
 - `bg` far riprendere l'esecuzione in background

Segnali nella shell

Molto utile se ho lanciato un comando lungo e voglio usare la shell mentre esegue

```
$ ./longjob
^Z
[1]+  Stopped                  ./longjob
$ bg
[1]+  ./longjob &
$ terminale libero
```

Segnali nella shell

Quando eseguo un programma in background (`./job &`) e chiudo il terminale, viene mandato il segnale di Hang Up `SIGHUP`

- Di default il programma viene terminato
- Si può modificare comportamento

Oppure uso il comando `nohup` che esegue un comando immune a `SIGHUP`

```
nohup ./job
```

Utile se lancio job su terminale remoto e devo andare a casa!

Alternativa più pulita: comando `screen` che genera terminale virtuale

Segnali nella shell

Handler di segnali in script bash

```
trap command SIGNAL
```

Esegue il comando o la funzione `command` se lo script riceve il segnale `SIGNAL`

Esiste lo pseudo-segnale aggiuntivo `EXIT`, chiamato quando lo script termina

Esempio tipico:

```
tempfile=/tmp/tmpdata  
trap "rm -f $tempfile" EXIT
```

Segnali nella shell

Esercizio: si crei un programma bash che conta quanti SIGUSR2 riceve, e li stampa quando viene premuto `CTRL+C` e lo si nomina `sample.sh`

```
#!/bin/bash

count=0
function husr(){
    let count++
}
function hint(){
    echo "Ricevuti $count SIGUSR2"
    exit 0
}

trap husr SIGUSR2
trap hint SIGINT

while true; do
    sleep 1
done
```

Si inviino i segnali col comando: `bash pkill -USR2 sample.sh`

Nota: dichiarazione di funzione in Bash

Domande

Quale System Call si usa per generare un segnale?

- `signal`
- `kill`
- `write`
- `send`

Una funzione handler riceve degli argomenti?

- No
- Riceve una stringa
- Riceve un intero

Quale è il comportamento di default di un processo quando riceve un segnale?

- Il segnale viene ignorato
- Il processo termina
- Dipende dal segnale

Un signal handler può modificare le variabili globali del processo?

- Si
- No

Quale segnale viene inviato dal SO quando si preme `CTRL+C` sulla tastiera?

- `SIGKILL`
- `SIGINT`
- `SIGHUB`
- `SIGSTP`

Sistemi Operativi
Unità 5: I processi
Inter-Process Communication

Martino Trevisan
Università di Trieste
Dipartimento di Ingegneria e Architettura

Argomenti

1. Obiettivi
2. Le *pipe*
3. Le *FIFO*
4. Cenni di memoria virtuale
5. Memoria condivisa con `mmap`
6. Problematiche

Obiettivi

Obiettivi

In un sistema dotato in SO, diversi processi sono in esecuzione contemporaneamente.

Essi possono essere classificati in:

- **Processi indipendenti:** non sono influenzati né influenzano altri processi
- **Processi cooperanti:** interagiscono con altri processi. Devono usare meccanismi opportuni per farlo

Obiettivi

Tutti i SO mettono a disposizione strumenti per la **Inter-Process Communication**

Sono tipicamente basati su:

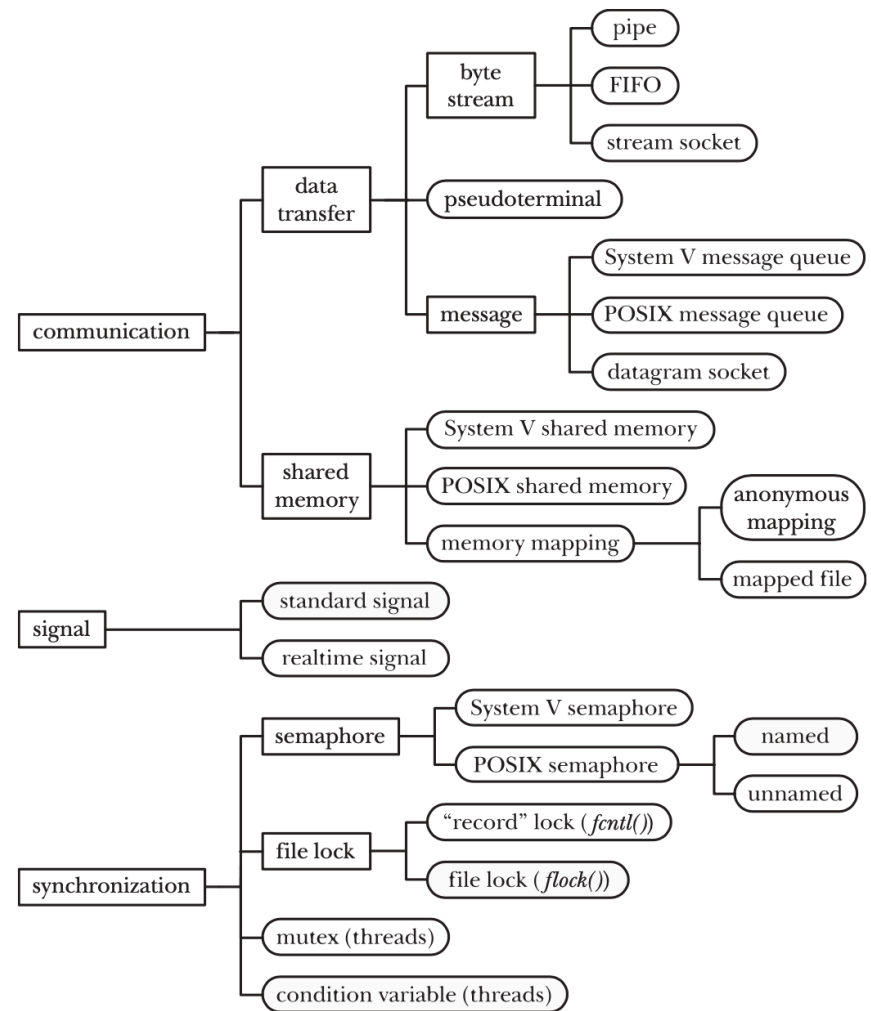
- **Scambio di messaggi**
- **Scambio di dati**
- **Memoria condivisa**

Obiettivi

Ogni SO utilizza meccanismi diversi.

In Linux, ci sono tanti meccanismi.

- Storicamente stratificati
- Ereditati da System V
- Parte di standard POSIX



Obiettivi

Noi vediamo:

- **Scambio di messaggi:** Segnali (già visto)
- **Scambio di dati:** *Pipe* e *FIFO*
- **Memoria condivisa:** utilizzo di System Call `mmap`

Le pipe

Le *pipe*

Le *pipe* sono la più vecchia e la più usata forma di IPC introdotta in Unix

- Permettono di scambiare dati tra processi
- Modello **produttore-consumatore**
- Si usano con le stesse System Call dei file: `read` , `write`
- Risiedono in memoria
- Non sono persistenti: quando i processi terminano, tutto ciò che rimane viene distrutto

Le *pipe*

Limitazioni:

- Sono half-duplex (comunicazione in un solo senso)
- Utilizzabili solo tra processi con un "antenato" in comune

Come superare queste limitazioni?

- Le *FIFO* (o *named pipe*) possono essere utilizzati tra più programmi
 - Si identificano tramite un nome

Le *pipe*

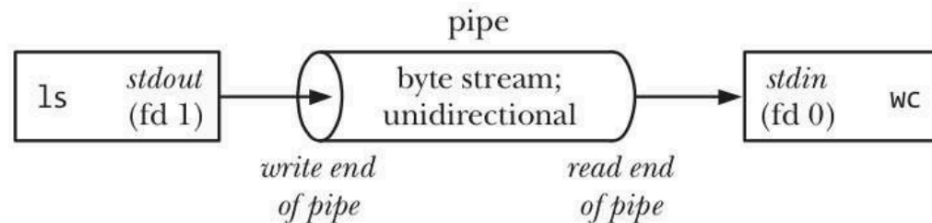
Le *pipe* sono comunemente usate nella shell, per redirezionare gli `stdout` e `stdin`.

Esempio:

```
ls | wc -l
```

Per fare questa operazione, la shell:

- Usa due `fork` e `exec` per creare io processi `ls` e `wc`
- Crea una *pipe* per connettere lo `stdout` di `ls` con lo `stdin` di `wc`



Le *pipe*

Definizione:

Le pipe sono un **byte stream**

- Vi si scrivono/leggono byte
- Non solo caratteri stampabili

Sono **unidirezionali**:

- Hanno un ingresso e una uscita

Hanno **capacità limitata**:

- I dati accodati (scritti ma non ancora letti) non possono eccedere una soglia
- Soglia configurabile: 65 KB di default
 - Si può variare con `fcntl(fd, F_SETPIPE_SZ, size)`

Le *pipe*

Creazione:

```
#include <unistd.h>
int pipe (int fildes [2]);
```

Ritorna due descrittori di file attraverso l'argomento fd (passato per riferimento)

- `fd[0]` è aperto in lettura
- `fd[1]` è aperto in scrittura
- L'output di `fd[1]` è l'input di `fd[0]`

Le *pipe*

I/O su *pipe*:

Si usano le funzioni `read` e `write`

- Il valore di ritorno è il numero di byte scritti/letti

Lettura:

- La `read` è bloccante finchè non è letto almeno un byte

Scrittura:

- Se la *pipe* è piena, la `write` è bloccante

Le *pipe*

Chiusura di una pipe:

La `read` blocca finchè non viene letto almeno 1 byte

Se la `read` ritorna `0`, vuol dire che non c'è nessun `fd` aperto in scrittura

- La *pipe* è morta

Le *pipe*

Nota: se scrivo su una *pipe* che non ha un *reader* (`fd[0]` è stato chiuso), il processo riceve il segnale `SIGPIPE` (*broken pipe*)

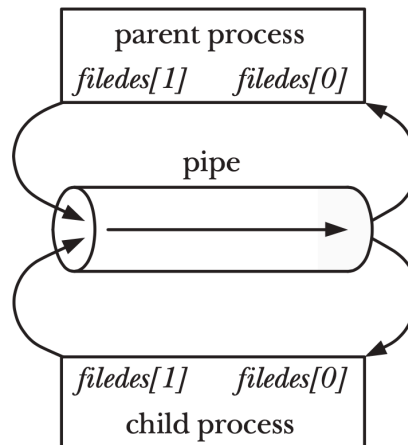
Terminazione del processo, se non c'è un **Signal Handler** opportuno

Le pipe

Condivisione tra processi:

Per utilizzare una pipe tra più processi:

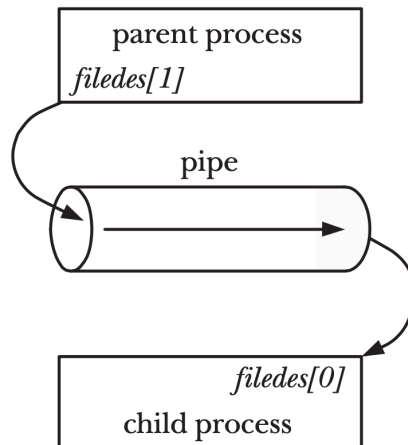
- Il processo padre crea la pipe e ottiene i due `fd`
- Esso fa una `fork`
- Entrambi i processi possono accedere alla *pipe* usando i due `fd`



Le pipe

Chiusura di una pipe:

- Solitamente un processo (e.g., padre) scrive, e un altro (e.g., figlio) legge
- Tecnicamente possibile che un processo legga e scriva
 - Crea però problemi di sincronizzazione
- Ogni processo chiude i `fd` che non usa



Le pipe

Esempio:

```
int pfd [2] ;

pipe ( pfd ); /* Crea la pipe */

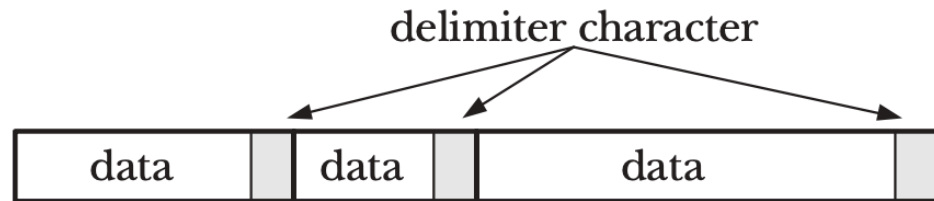
if ( fork () == 0 ) { /* Figlio */
    close ( pfd [1]);
    /* Legge
        ... */
} else { /* Padre */
    close ( pfd [0]);
    /* Scrive
        ... */
}
```


Le *pipe*

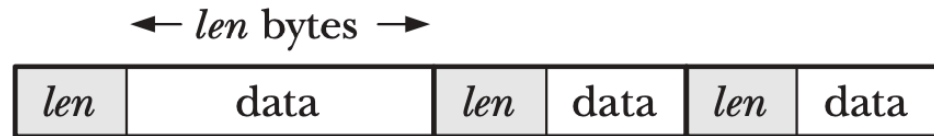
Messaggi su pipe

Ci sono diverse strategie per scambiare messaggi tramite *pipe*

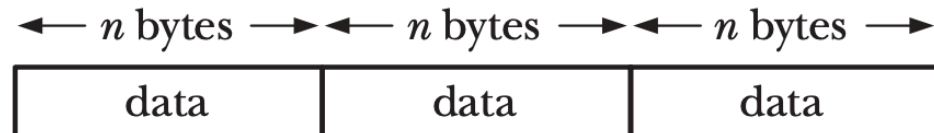
1) delimiter character



2) header with length field



3) fixed-length messages



Le pipe

Esercizio: si crei un programma che genera un figlio. Il processo padre riceve una stringa da riga di comando e la passa al figlio tramite una *pipe*. Il figlio riceve la stringa e la stampa.

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>
#define MAXLINE 1024

int main(int argc, char *argv[])
{
    int pfd[2], status;
    char line[MAXLINE];

    pipe(pfd);
    if (fork() > 0) { /* Padre */
        close(pfd[0]);
        write(pfd[1], argv[1], strlen(argv[1]));
        wait(&status);
    } else { /* Figlio */
        close(pfd[1]);
        read(pfd[0], line, MAXLINE);
        printf("Ricevuto: %s\n", line);
    }
    exit(0);
}
```

Le pipe

Esercizio: si crei un programma con due processi. Il processo padre riceve il nome di un file da riga di comando e ne passa il contenuto al figlio tramite una *pipe*. Il figlio riceve il contenuto e lo stampa.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#define MAXSIZE 1000

int main(int argc, char * argv[]){
    int pfd[2], fp, status;
    char buffer [MAXSIZE];
    pipe(pfd); /* Crea la pipe */
    if(fork()>0){
        close(pfd[0]); /* Chiude capo di scrittura */
        fp=open(argv[1],O_RDONLY);
        while(read(fp, buffer,sizeof(buffer))>0) /* Itera riga per riga*/
            write(pfd[1],buffer,sizeof(buffer)); /* Scrive nella pipe */
        close(pfd[1]);
        close (fp);
        wait(&status);
        exit(0);
    }
    else {
        close(pfd[1]); /* Chiude capo di lettura */
        while (read(pfd[0],buffer,sizeof(buffer)) > 0) /* Quando read ritorna 0, la pipe è morta*/
            printf("%s", buffer);
        close(pfd[0]);
        exit(0);
    }
}
```

Le *FIFO*

Le *FIFO*

Pipe "normali"

- Possono essere utilizzate solo da processi che hanno un "antenato" in comune
- Motivo: unico modo per ereditare descrittori di file

Named pipe o *FIFO*

- Permettono a processi non collegati di comunicare
- Utilizzano il file system per "dare un nome" alla pipe
- Le *FIFO* sono un tipo di *nodo*
 - La macro `S_ISFIFO` dopo una `stat` restituirà `true`
- La procedura per creare una `fifo` è simile alla procedura per creare file

Le *FIFO*

```
#include <sys/types.h>
#include <sys/stat.h>
int mkfifo(const char *path, mode_t mode);
```

Crea un *FIFO* dal `pathname` specificato

Argomento `mode` specifica i permessi (come nella `open`).

- E.g.: `S_IRWXU`, `S_IRGRP`, etc.

Valore di ritorno: `0` se successo, `-1` se errore

Utilizzo:

Come file e *pipe*: tramite `read` e `write`

Ogni processo che ha i permessi per `pathname` può usarla

Le *FIFO*

Apertura: dopo essere state create con `mkfifo`, le FIFO vanno aperte con una `open` o una `fopen`

La `open` è bloccante finchè la FIFO non ha un produttore e un consumatore

- Se il file è aperto in lettura, la `open` si blocca fino a quando un altro processo non apre la FIFO in scrittura
- Se il file è aperto in scrittura, la `open` si blocca fino a quando un altro processo non apre la FIFO in lettura

Le *FIFO*

Input/Output:

Con `read` e `write`

I dati in transito nella *FIFO* sono nella memoria del kernel

Importante:

Una *FIFO* ha un pathname ma è solo un espediente per permettere a diversi processi di accedervi

Quando un *FIFO* viene chiusa (o i processi terminano) il nome del file persiste nel file system, ma esso non contiene alcun dato

Le *FIFO*

Si possono creare e usare le *FIFO* in Bash in maniera semplice:

```
mkfifo myfifo  
tr 'aeiou' 'AEIOU' < myfifo &  
man 2 pipe > myfifo
```

Esercizio: si crei un programma che: 1) se riceve `read` come argomento, stampa ciò che viene scritto su una FIFO e 2) se riceve `write` come argomento, legge iterativamente una riga da tastiera e la scrive su una FIFO.

```
#include <stdio.h>
#include <sys/stat.h>
#include <string.h>
#include <stdlib.h>
#define FIFO "my-fifo"
#define BUF_SIZE 512

int main(int argc, char * argv[]){
    FILE * f;
    char buffer[BUF_SIZE];

    if (argc != 2 || ( strcmp(argv [1], "read")==0 && strcmp(argv [1], "write")==0 ) ){
        printf("Usage: fifo read|write\n");
        return 1;
    }

    if (mkfifo(FIFO, S_IRWXU)<0)
        perror("Warning. FIFO not created");

    if (strcmp(argv [1], "read")==0){
        f = fopen(FIFO, "r");
        if (f==NULL){
            perror("Impossible to open the FIFO");
            return 1;
        }

        printf("Read mode:\n");
        while(fgets(buffer, BUF_SIZE, f)!=NULL)
            printf("%s", buffer);
    }else{
        f = fopen(FIFO, "w");
        if (f==NULL){
            perror("Impossible to open the FIFO");
            return 1;
        }

        printf("Write mode. Write lines of text:\n");
        while(fgets(buffer, BUF_SIZE, stdin)!=NULL){
            fputs(buffer, f);
            fflush(f);
        }
    }
    return 0;
}
```

Le FIFO

Esercizio: si crei un programma che legge da una FIFO e stampa il contenuto in maiuscolo.

```
#include <stdio.h>
#include <ctype.h>
#include <sys/stat.h>
#include <string.h>
#include <stdlib.h>

int main(int argc, char * argv[]){
    int i, n, l;
    FILE * f;
    char buffer[512];

    i = mkfifo("myfifo", S_IRWXU);
    if (i<0){
        printf("Impossibile creare la FIFO\n"); /* Potrebbe già esistere */
    }

    f = fopen("myfifo", "r");
    if (f==NULL){
        printf("Impossibile aprire la FIFO\n");
        exit(1);
    }
    while(fgets(buffer, sizeof(buffer), f)!=NULL){
        l = strlen(buffer);
        for (i=0; i<l; i++)
            putc(toupper(buffer[i]), stdout);
    }
}
```

La si testi con: `echo "ciao mondo" > myfifo`

Cenni di memoria virtuale

Cenni di memoria virtuale

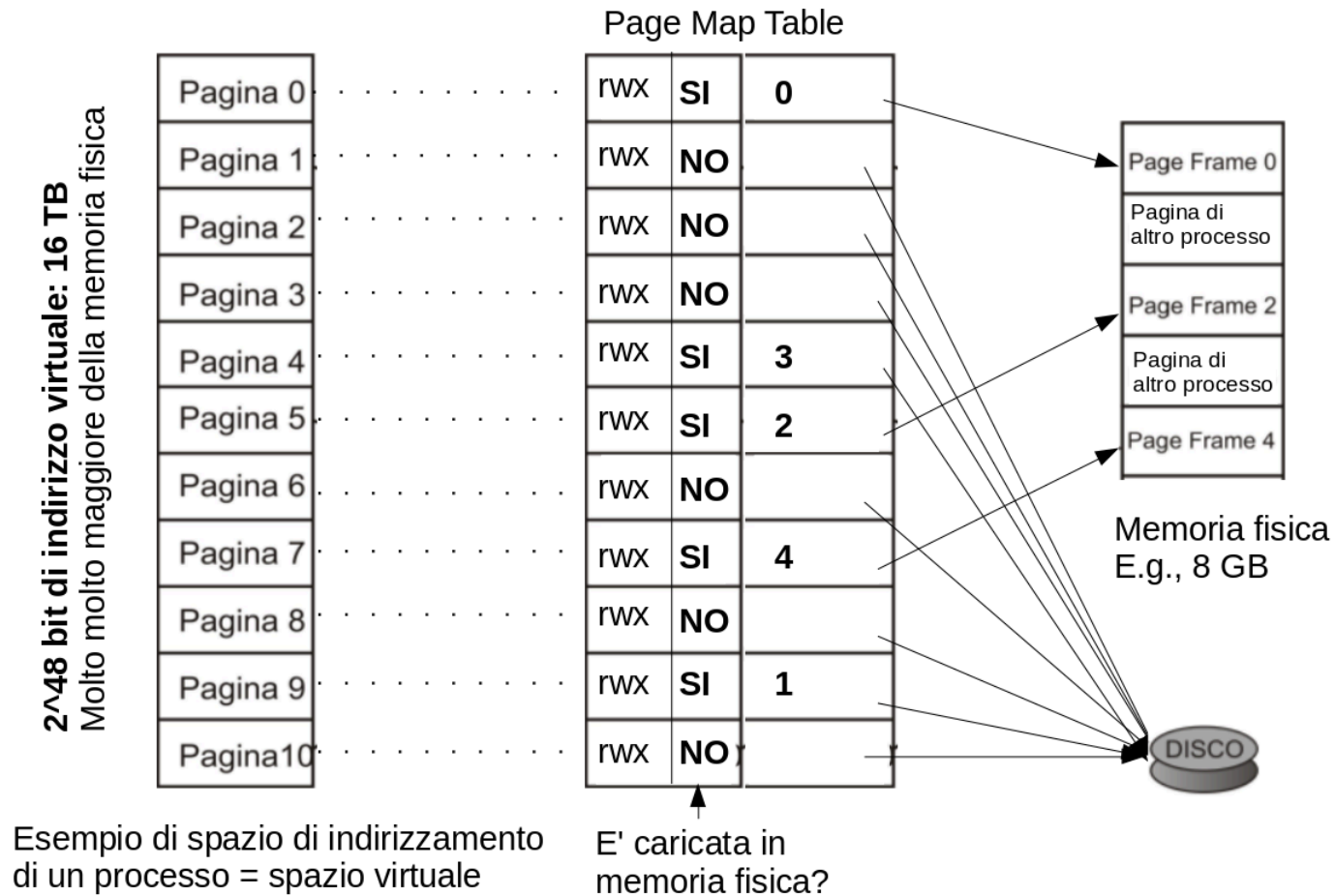
I processi emettono indirizzi virtuali

- Permettono di indirizzare più memoria di quella disponibile
 - Su architettura AMD64: 48bit; 256TB di memoria virtuale. La memoria fisica è di solito minore (e.g., 16 GB)
- Evitano che un processo acceda a memoria di altri

La memoria è divisa in **pagine** e una tabella mappa le pagine da spazio di indirizzi virtuali a indirizzi fisici

- Azione compiuta dalla **Memory Management Unit** in Hardware
 - Il sistema operativo interviene a collocare pagine in memoria

Cenni di memoria virtuale



Cenni di memoria virtuale

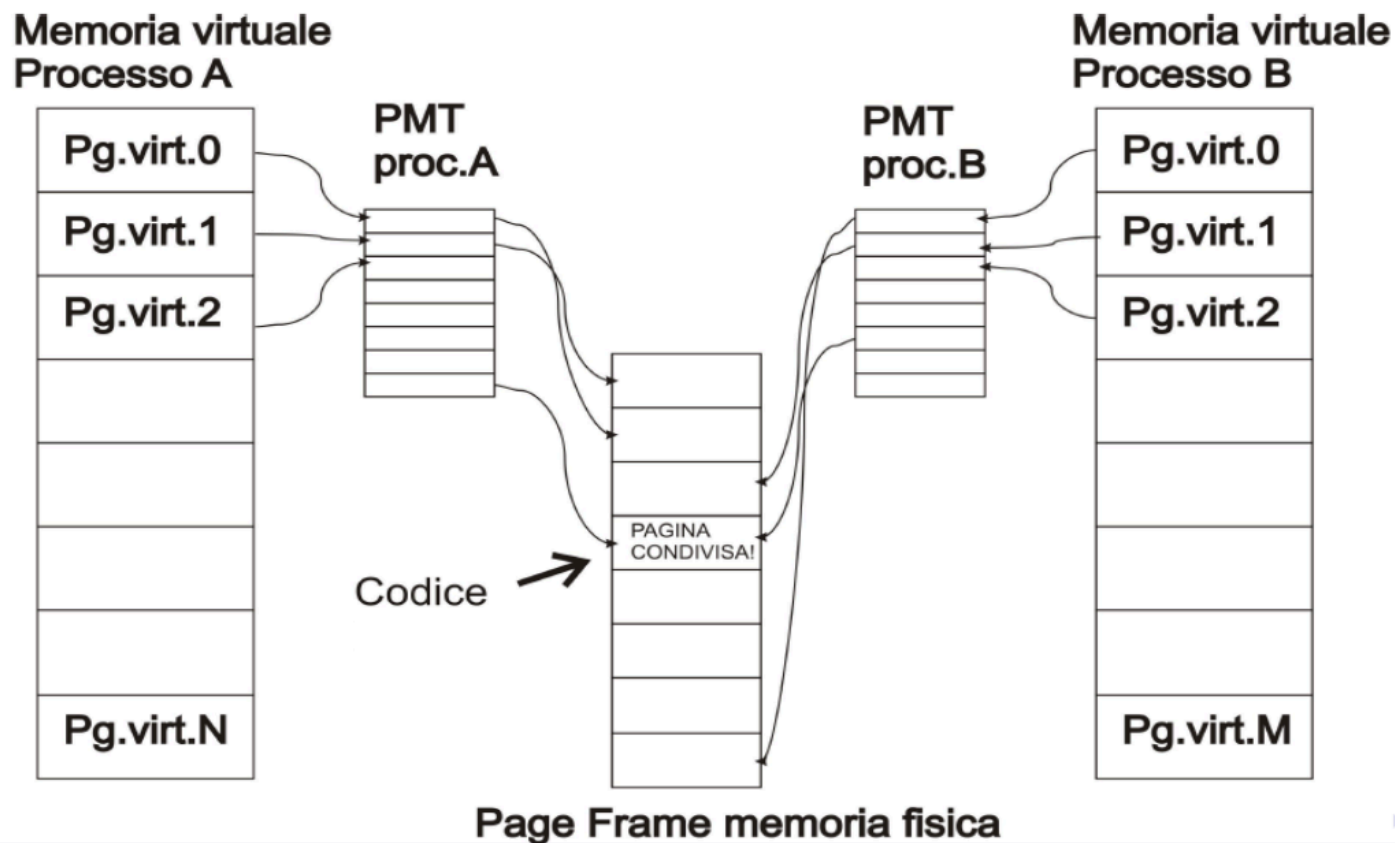
Ogni processo ha uno spazio di indirizzi virtuali dedicato

- C'è una tabella delle pagine per processo
- Isolamento della memoria tra processi
 - Essenziale per sicurezza
 - Non permette la condivisione di memoria

Per condividere la memoria, è necessario condividere una o più pagine

- Il SO mette a disposizione delle System Call per questo scopo

Cenni di memoria virtuale



Cenni di memoria virtuale

Esistono due set di System Call per avere memoria condivisa tra processi in Linux:

- `shmget shmat shmdt ftok` (non vediamo)
- `mmap munmap shm_open shm_unlink` (**vediamo**)

L'approccio con `mmap` è più moderno e flessibile

In Windows si usa la System Call `CreateFileMapping`

Memoria condivisa con `mmap`

Memoria condivisa con `mmap`

```
#include <sys/mman.h>
void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

Crea una zona di memoria condivisa.

Argomenti:

- `addr` : se non nullo, la memoria viene mappata a `addr` (arrotondato per difetto al *page size*)
- `length` : dimensione
- `prot` : può essere: `PROT_READ` , `PROT_WRITE` , `PROT_EXEC` , `PROT_NONE`
 - Cioè la pagina può essere letta, scritta, eseguita, non può essere acceduta
 - Normalmente `prot = PROT_READ|PROT_WRITE`
- `flags` determina come i cambiamenti sono visibili o meno ad altri processi
 - `MAP_ANONYMOUS`, `MAP_SHARED`, `MAP_PRIVATE`
 - Normalmente `flags=MAP_ANONYMOUS | MAP_SHARED` o `MAP_PRIVATE`
- `fd` e `offset` sono usati per mappare la memoria su un file

Memoria condivisa con `mmap`

```
#include <sys/mman.h>
void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

Valore di ritorno:

- L'indirizzo virtuale del segmento mappato
- `-1` se insuccesso

```
int munmap(void *addr, size_t length);
```

Rimuove la mappatura e rende disponibile la memoria all'indirizzo `addr`

Memoria condivisa con `mmap`

Utilizzo: ci sono tre modi per usare la `mmap`

1. Zona di memoria anonima: utilizzata con `fork`

```
mmap(NULL, size, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, -1, 0);
```

2. Zona di memoria mappata su file:

```
fd = open("/home/martino/file.txt", O_RDWR|O_CREATE);  
mmap(NULL, size, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

3. Zona di memoria mappata su file temporaneo:

```
fd = shm_open("temporaneo.txt", "rw");  
mmap(NULL, size, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

Memoria condivisa con `mmap`

1. Zona di memoria anonima:

- La zona di memoria non ha un nome
- Solo un figlio nato con una `fork` può accedervi
- Semplicissimo da usare

```
void* shmem = mmap(NULL, size, PROT_READ | PROT_WRITE,  
                  MAP_SHARED | MAP_ANONYMOUS, -1, 0);  
  
if (fork()){ /* Padre */  
    ...  
} else { /* Figlio */  
    ...  
}  
munmap(shmem, size);
```

Nota: Dopo la `fork`, la memoria dei processi è indipendente. Solo con `mmap` è possibile creare una zona di memoria condivisa.

Memoria condivisa con `mmap`

2. Zona di memoria mappata su file:

- Si usa un file come contenitore
- Il contenuto della zona condivisa verrà salvato su file
- Più programmi possono accedere alla memoria condivisa
 - Il path va identificatore
- Efficace ma lento a causa del disco

Memoria condivisa con `mmap`

2. Zona di memoria mappata su file:

E' necessario che il file sia grande a sufficienza per contenere la regione mappata

```
#include <unistd.h>
int truncate(const char *path, off_t length);
int ftruncate(int fd, off_t length);
```

Assicura che il file aperto `fd` o il file `path` si lungo almeno `length` .

- Se necessario il file è troncato
- Se necessario esteso e riempito con caratteri `'\0'` (o `0x00` in hex).

Memoria condivisa con `mmap`

2. Zona di memoria mappata su file:

Flusso tipico:

```
fd = open(path, O_RDWR|O_CREATE); /* Non è una fopen() */
ftruncate(fd, size);
void* shmem = mmap(NULL, size, PROT_READ|PROT_WRITE,
                  MAP_SHARED, fd, 0);

...
munmap(shmem, size);
```

Memoria condivisa con `mmap`

3. Zona di memoria mappata su file temporaneo

- A volte è necessario avere un file temporaneo identificabile
 - Per utilizzo da parte di più programmi
- Una zona di memoria mappata su file è inutilmente lenta
 - Se non è necessario che i dati della zona di memoria sopravvivano

Si possono utilizzare le funzioni `shm_open` e `shm_unlink`

- Creano e rimuovono un file temporaneo
- Che si trova nella cartella `/dev/shm/` che ha montato un FS temporaneo (`tmpfs`)
- I dati sono **in memoria**

Memoria condivisa con `mmap`

3. Zona di memoria mappata su file temporaneo

```
#include <sys/mman.h>
int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);
```

Creano e rimuovono zone di memoria temporanee.

- Semantica analoga a `open` e `unlink`
- Operano su zone di memoria/file temporanee
- `name` è un nome di file, non un path completo
 - Tutte le zone di memoria sono file sotto `/dev/shm/`

Memoria condivisa con `mmap`

3. Zona di memoria mappata su file temporaneo

Flusso tipico

```
int fd = shm_open(nome, O_RDWR, 0);
ftruncate(fd, size);
void * mem = mmap(NULL, size, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);

...
munmap(shmem, size);

/* Non obbligatorio */
shm_unlink(nome)
```

Memoria condivisa con `mmap`

Esercizio: Si creino due programmi che hanno una memoria condivisa con `mmap` e `shm_open`.

Il primo programma permette di scrivere una stringa nella memoria, mentre il secondo permette di leggerla.

Programma 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <fcntl.h> /* Per O_RDWR */
int main(){
    int fd;
    char * mem;

    fd = shm_open("mymem", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
    ftruncate(fd, 512);
    mem = (char *) mmap(NULL, 512, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    while(1){
        printf("Scrivere: ");
        scanf("%s", mem);
    }
    munmap(mem, 512); /* Inutile causa loop infinito*/
}
```

Memoria condivisa con `mmap`

Programma 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <fcntl.h> /* Per O_RDWR */
int main(){
    int fd;
    char * mem;

    fd = shm_open("mymem", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
    ftruncate(fd, 512);
    mem = (char *) mmap(NULL, 512, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    while(1){
        printf("Premi enter per leggere");
        getchar();
        printf("Data: %s\n\n", mem);
    }
    munmap(mem, 512);
}
```

Nota: Compilare con `gcc prog.c -lrt -o prog`

Include `librt, libposix4` - POSIX.1b Realtime Extensions library

Problematiche

Problematiche

L'utilizzo della memoria condivisa è complesso

- Preferire le *pipe* o *FIFO* quando possibile

La memoria condivisa ha problemi di sincronizzazione e *race conditions*

- Processi concorrenti possono leggere dati in stato inconsistente
- Mentre un'altro processo li stava modificando
- Come abbiamo già visto coi segnali

Problematiche

Esempio:

Time	τ_x	τ_y
t_1	READ (A)	—
t_2	$A = A - 50$	
t_3	—	READ (A)
t_4	—	$A = A + 100$
t_5	—	—
t_6	WRITE (A)	—
t_7		WRITE (A)

LOST UPDATE PROBLEM

La variabile A viene incrementata di 100.

- Il decremento di 50 viene perso

Problematiche

Per ovviare a questi problemi esistono le **tecniche di sincronizzazione**

- Permettono di evitare che un processo sia interrotto mentre effettua un'operazione critica
- Permettono a un processo di attendere il verificarsi di una condizione

Vedremo più avanti

Domande

Le *pipe* sono

- Monodirezionali
- Bidirezionali
- Dipende dai parametri di creazione

Le *pipe* sono identificate da un nome?

- Si
- No

Le *FIFO* sono identificate da?

- Un ID numerico
- Una stringa``
- Un Path

Si consideri il seguente codice C che opera sulla FIFO `myfifo` :

```
int n;  
FILE * f = fopen("myfifo", "r");  
fscanf (f, "%d", &n);
```

Che operazione compie?

- Crea la FIFO `myfifo`
- Scrive un intero in `myfifo`
- Legge un intero da `myfifo`

Domande

Una zona di memoria condivisa creata tramite `shmget` e `shmat` può essere condivisa anche tra processi senza legami di parentela?

- Si
- No

Le funzioni `shmat` e `mmap` hanno valore di ritorno:

- `char *`
- `int`
- `int*`
- `void*`
- `void`

Si consideri il seguente spezzone di codice:

```
int fd = open("/tmp/mymem", O_RDWR|O_CREATE);
ftruncate(fd, 64);
void* shmem = mmap(NULL, size, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
sprintf((char*)shmem, "Ciao Mondo!");
```

Dopo che il programma è terminato, il contenuto della zona di memoria persiste?

- Si, in memoria
- Si, nel file `/tmp/mymem`
- No